

Python Practice Workbook

Module 3: Forecasting & Model Selection

AIC/BIC, Diagnostics, Unit Root, Holt-Winters, Forecast Evaluation

Time Series Analytics T6725 — M.Sc. Data Science Sem IV

Symbiosis International (Deemed) University

Primary Dataset: BTC-USD — Supporting: GC=F, MRF.NS

Standard Setup Block — Run this first in every Colab session

```
!pip install yfinance statsmodels arch -q
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

Problem Distribution

Difficulty	Count	Types Used
EASY	10	FIB, FTE, MCQ, PTO, SA
MEDIUM	12	FIB, FTE, GOR, MTF, MCQ, WFS
HARD	12	FIB, FTE, MCQ, WFS, PTO, SA
EXAM	6	Multi-part
TOTAL	40	All 8 types

Section A

Easy Problems — M3.001 to M3.010

M3.001 Fill in the Blank **EASY**

Topic: Holt-Winters — Triple Exponential Smoothing setup

Fill in the **five blanks** to fit a Holt-Winters model to monthly Gold prices with additive trend and multiplicative seasonality.

```

1 from statsmodels.tsa.holtwinters import -----
2
3 gold      = yf.download("GC=F", start="2019-01-01",
4                       end="2024-12-31")
5 monthly   = gold["Close"].dropna().resample("-----").last()
6
7 model = ExponentialSmoothing(
8     monthly,
9     trend=        "-----",
10    seasonal=      "-----",
11    seasonal_periods= -----
12)
13 fit = model.fit(optimized=True)
14 print(fit.summary())
15 print(f"\nSmoothing level (alpha):
16       {fit.params['smoothing_level']:.4f}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.holtwinters import
6     ExponentialSmoothing
7
8 gold      = yf.download("GC=F", start="2019-01-01",
9                       end="2024-12-31")
10 monthly   = gold["Close"].dropna().resample("ME").last()  #
11           "ME" = Month End
12
13 model = ExponentialSmoothing(
14     monthly,
15     trend=        "add",  # additive: trend
16                       grows by fixed amount
17     seasonal=      "mul",  # multiplicative:
18                       swings scale with level
19     seasonal_periods= 12    # 12 months = 1
20                           complete seasonal cycle

```

```

15         )
16 fit = model.fit(optimized=True) # finds best alpha, beta,
    gamma
17 print(fit.summary())
18 print(f"\nSmoothing level (alpha):
    {fit.params['smoothing_level']:.4f}")

```

② Plain English

Holt-Winters extends simple exponential smoothing with two extra components: a trend smoother (β) and a seasonal smoother (γ). `trend="add"` means the trend adds a fixed increment each period. `seasonal="mul"` means seasonal swings are proportional to the current level — appropriate for Gold whose price swings grow as the price rises.

③ Common Mistake

Using `seasonal_periods=365` on monthly data — the period is the number of *data points* per cycle, not calendar days. Monthly data with annual seasonality = 12 data points per year = `seasonal_periods=12`. Using 365 on monthly data would look for a 30-year cycle.

④ Real World Link

Gold's annual seasonal pattern (Diwali demand, Chinese New Year, summer doldrums) makes Holt-Winters a natural first-pass forecasting model. The multiplicative seasonal component handles the fact that a 3% Diwali bump is worth \$60 when gold is at \$2000/oz but only \$38 when gold was at \$1280/oz in 2019.

M3.002 Fill in the Blank **EASY**

Topic: Generating forecasts and confidence intervals

Fill in the blanks to forecast BTC prices 30 days ahead and extract Monte Carlo confidence intervals.

```

1 from statsmodels.tsa.holtwinters import
    ExponentialSmoothing
2
3 btc    = yf.download("BTC-USD", start="2019-01-01",
    end="2024-12-31")
4 close  = btc["Close"].dropna()
5
6 model  = ExponentialSmoothing(close, trend="add",
    seasonal=None)
7 fit    = model.fit(optimized=True)
8
9 # Forecast 30 steps ahead
10 forecast = fit._____( _____ )
11
12 # Monte Carlo confidence intervals via simulation
13 simulations = fit.simulate(
14     nsimulations= _____,

```

```

15         repetitions = 500,
16         error       = "add"
17     )
18
19     lower = simulations.quantile(_____, axis=1)  # 2.5th
           percentile
20     upper = simulations.quantile(_____, axis=1)  # 97.5th
           percentile
21
22     print(f"Point forecast Day 30: ${forecast.iloc[-1]:,.2f}")
23     print(f"95% CI Day 30: [${lower.iloc[-1]:,.2f},
           ${upper.iloc[-1]:,.2f}]")

```

Solution

① Correct Code

```

1  !pip install yfinance statsmodels arch -q
2  import yfinance as yf
3  import warnings
4  warnings.filterwarnings('ignore')
5  from statsmodels.tsa.holtwinters import
     ExponentialSmoothing
6
7  btc    = yf.download("BTC-USD", start="2019-01-01",
8                      end="2024-12-31")
9  close  = btc["Close"].dropna()
10
11  model = ExponentialSmoothing(close, trend="add",
12                               seasonal=None)
13  fit    = model.fit(optimized=True)
14
15  # Forecast 30 steps ahead -- .forecast() returns a pandas
16  # Series
17  forecast = fit.forecast(30)
18
19  # Simulate 500 future paths for Monte Carlo confidence
20  # intervals
21  simulations = fit.simulate(
22      nsimulations=30,    # 30 steps ahead
23      repetitions=500,    # 500 random paths
24      error="add"
25  )
26
27  lower = simulations.quantile(0.025, axis=1)  # 2.5th
           percentile -> lower 95% CI
28  upper = simulations.quantile(0.975, axis=1)  # 97.5th
           percentile -> upper 95% CI

```

```

25
26 print(f"Point forecast Day 30: ${forecast.iloc[-1]:,.2f}")
27 print(f"95% CI Day 30: [{lower.iloc[-1]:,.2f},
    ${upper.iloc[-1]:,.2f}]" )

```

② Plain English

Holt-Winters `.forecast()` gives point forecasts only. For uncertainty quantification, `.simulate()` generates hundreds of hypothetical future paths by adding random noise to the forecast — the spread of those paths gives you a confidence interval. The 2.5th and 97.5th percentiles bracket the central 95% of outcomes.

③ Common Mistake

Confusing `.forecast(30)` and `.predict(30)`. In statsmodels Holt-Winters, `.forecast(n)` generates n steps ahead from the end of training data. `.predict(start, end)` generates in-sample fitted values plus optional out-of-sample. For pure forecasting beyond the data, always use `.forecast()`.

④ Real World Link

A BTC 30-day confidence interval from Holt-Winters will be extremely wide. If Day-30 point forecast is \$65,000 but the 95% CI runs from \$40,000 to \$90,000, that's statistically honest — anyone who gives you a narrow BTC price forecast is hiding uncertainty, not eliminating it.

M3.003 Multiple Choice **EASY**

Topic: AIC penalty term

The AIC formula is $AIC = 2k - 2\log(L)$ where k is the number of parameters and L is the maximised likelihood. If you add one extra parameter and the log-likelihood improves by exactly 0.8, what happens to AIC?

- (A) AIC decreases by 0.4 — the model improves
- (B) AIC increases by 0.4 — the extra parameter costs more than it gains
- (C) AIC stays the same — gain and penalty exactly cancel
- (D) AIC decreases by 1.6 — log-likelihood gain always dominates

Solution

① Correct Answer: (B)

② Plain English

Adding one parameter increases the penalty term $2k$ by exactly 2. The improvement in $-2\log(L)$ is $-2 \times 0.8 = -1.6$ (negative because better likelihood lowers this term). Net change in AIC = $+2 - 1.6 = +0.4$. Since lower AIC is better, the extra parameter made the model slightly worse by AIC — it didn't justify its complexity cost.

③ Common Mistake

Forgetting the factor of 2 in the AIC formula. The penalty is $2k$ (not just k) and the likelihood term is $-2\log(L)$ (not just $-\log(L)$). Always apply the formula carefully:

$$\Delta\text{AIC} = 2\Delta k - 2\Delta \log(L) = 2(1) - 2(0.8) = +0.4.$$

④ Real World Link

This calculation tells you the minimum log-likelihood improvement needed to justify each added parameter: you need $\Delta \log(L) > 1$ per extra parameter for AIC to favour the more complex model. For BTC and Gold models where sample size is large ($n > 1000$), even tiny improvements matter — small AIC differences of 0.5–2 units are genuine signals.

M3.004 Multiple Choice **EASY**

Topic: BIC vs AIC — sample size sensitivity

You have two datasets: Dataset A ($n = 50$ monthly Gold observations), Dataset B ($n = 1250$ daily BTC observations). For which dataset will BIC penalise model complexity MORE aggressively relative to AIC?

- (A) Dataset A ($n = 50$) — smaller samples need more protection from overfitting
- (B) Dataset B ($n = 1250$) — BIC penalty grows with $\log(n)$
- (C) Both datasets — BIC always applies the same penalty as $2 \times \text{AIC}$
- (D) Neither — AIC and BIC give identical rankings when $n > 30$

Solution

① **Correct Answer:** (B)

② **Plain English**

BIC's penalty is $k \log(n)$ versus AIC's $2k$. For Dataset A: $\log(50) \approx 3.9$, so BIC penalises each parameter $3.9 \times$ vs AIC's $2 \times$. For Dataset B: $\log(1250) \approx 7.1$, so BIC penalises $7.1 \times$ vs AIC's $2 \times$. The BIC-to-AIC ratio grows with $\log(n)$ — so BIC is proportionally stricter for *larger* datasets.

③ **Common Mistake**

Choosing (A) with the reasoning “small samples overfit more.” While true that small samples are more prone to overfitting, BIC's *relative extra penalty over AIC* actually increases with n . For $n = 50$: BIC penalises $3.9/2 = 1.95 \times$ more than AIC. For $n = 1250$: BIC penalises $7.1/2 = 3.55 \times$ more than AIC. Larger $n \rightarrow$ bigger BIC advantage.

④ **Real World Link**

When building a forecasting model for BTC daily returns ($n \approx 1250$), BIC will aggressively prefer very simple models — AR(1) over AR(2), ARMA(1,1) over ARMA(2,1). For Gold monthly data ($n \approx 60$), AIC and BIC often agree since the penalty difference is smaller. Always report both criteria and note if they disagree.

M3.005 Predict the Output **EASY**

Topic: Forecast error metrics — manual calculation

Without running this code, compute the exact output.

```

1 import numpy as np
2
3 actual    = np.array([100, 105, 98, 112, 95])
4 forecast  = np.array([102, 103, 100, 108, 97])
5
6 errors    = actual - forecast
7
8 mae       = np.mean(np.abs(errors))
9 mse       = np.mean(errors ** 2)
10 rmse      = np.sqrt(mse)
11 mape      = np.mean(np.abs(errors / actual)) * 100
12
13 print(f"Errors: {errors}")
14 print(f"MAE   : {mae:.2f}")
15 print(f"MSE   : {mse:.2f}")
16 print(f"RMSE  : {rmse:.2f}")
17 print(f"MAPE  : {mape:.2f}%")

```

Work out the exact values for all five print statements.

Solution

① Predicted Output

```

Errors:  [-2  2 -2  4 -2]
MAE   : 2.40
MSE   : 6.40
RMSE  : 2.53
MAPE  : 2.32%

```

```

1 # Working:
2 # errors = [100-102, 105-103, 98-100, 112-108, 95-97] =
   [-2, +2, -2, +4, -2]
3 #
4 # MAE   = mean(2, 2, 2, 4, 2) = 12/5 = 2.40
5 #
6 # MSE   = mean(4, 4, 4, 16, 4) = 32/5 = 6.40
7 #
8 # RMSE  = sqrt(6.40) = 2.53
9 #
10 # MAPE  = mean(| 2 /100|, |2/105|, | 2 /98|, |4/112|,
   | 2 /95|) x 100
11 #       = mean(2.000, 1.905, 2.041, 3.571, 2.105) =
   11.622/5 = 2.32%

```

② Plain English

Four metrics, four interpretations: MAE tells you the average absolute error in original units. MSE squares errors so large mistakes hurt more. RMSE converts MSE back to original units for easy interpretation. MAPE expresses error as a percentage — useful for comparing accuracy across assets of different price scales (BTC vs MRF vs Gold all measured in the same %).

③ Common Mistake

The MSE is 6.40, not 5.60 — errors squared are (4, 4, 4, 16, 4), summing to 32, divided by 5 = 6.40. Students sometimes forget to square the individual errors and instead square the sum. In an exam, always compute each squared error separately before averaging.

④ Real World Link

MAPE is popular for comparing Gold, BTC, and MRF forecasts in a single table — a 2.3% MAPE means the same quality of fit regardless of whether you're forecasting a 90,000 MRF share or a \$65,000 BTC price. But MAPE breaks when actual values are near zero (division error) — for return series close to zero, use MAE or RMSE instead.

M3.006 Find the Error **EASY**

Topic: Train-test split — temporal ordering violated

This code evaluates forecast accuracy but makes a critical error. Find it.

```

1 import numpy as np
2 from sklearn.model_selection import train_test_split      #
   ERROR IS HERE
3
4 btc    = yf.download("BTC-USD", start="2019-01-01",
5                       end="2024-12-31")
6 close  = btc["Close"].dropna()
7
8 # Split into train and test
9 train, test = train_test_split(close, test_size=0.2,
10                               random_state=42)
11
12 from statsmodels.tsa.holtwinters import
13     ExponentialSmoothing
14 fit      = ExponentialSmoothing(train, trend="add").fit()
15 forecast = fit.forecast(len(test))
16
17 rmse = np.sqrt(((test.values - forecast.values)**2).mean())
18 print(f"Test RMSE: ${rmse:,.2f}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
7
8 btc    = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
9 close = btc["Close"].dropna()
10
11 # FIX: Time series splits must respect temporal order --
   NEVER use random splits
12 # sklearn's train_test_split shuffles data randomly,
   breaking time ordering
13 # Future data leaks into training set -> artificially
   inflated accuracy
14
15 # CORRECT split: last 20% of TIME as test set
16 split_idx = int(len(close) * 0.8)
17 train     = close.iloc[:split_idx]    # chronologically
   first 80%
18 test      = close.iloc[split_idx:]   # chronologically
   last 20%
19
20 print(f"Train: {train.index[0].date()} to
   {train.index[-1].date()} ({len(train)} days)")
21 print(f"Test:  {test.index[0].date()} to
   {test.index[-1].date()} ({len(test)} days)")
22
23 fit       = ExponentialSmoothing(train,
   trend="add").fit(optimized=True)
24 forecast  = fit.forecast(len(test))
25
26 rmse = np.sqrt(((test.values - forecast.values)**2).mean())
27 print(f"Test RMSE: ${rmse:,.2f}")

```

② Plain English

Time series data has a fundamental rule: the future cannot inform the past. `sklearn.train_test_split` shuffles randomly, mixing 2024 data into the training set and 2019 data into the test set — then the model “predicts” the past using future information, giving impossibly good scores. The correct split uses the first $n\%$ as training and the last $(100 - n)\%$ as test, preserving chronological order.

③ Common Mistake

This is the single most common error in time series machine learning. Students who learned cross-validation on tabular data reflexively import `train_test_split` without realising it's fundamentally incompatible with time series. Even k-fold cross-validation must be replaced by time series cross-validation (walk-forward validation) for sequential data.

④ Real World Link

If you use random splits to backtest a BTC trading strategy and get 85% accuracy, that number is meaningless — your model used 2024 BTC data to “predict” 2020 prices. This is called data leakage, and it's why many backtested quant strategies look great in research but fail in live trading. Temporal splits are non-negotiable.

M3.007 Short Answer **EASY**

Topic: In-sample fit vs out-of-sample forecast

In 2 sentences, explain the difference between in-sample fit (training error) and out-of-sample forecast accuracy (test error). Why is it dangerous to select models based only on in-sample fit?

Solution

① Model Answer

In-sample fit measures how well a model explains the historical data it was trained on — a model can always improve in-sample fit by adding more parameters, even if those parameters capture noise rather than genuine patterns. Out-of-sample accuracy tests whether the model generalises to *new, unseen* data — and overfitted models that memorised training noise perform poorly on test data, revealing that their in-sample fit was illusory.

② Common Mistake

Thinking higher complexity always means better models because in-sample RMSE always decreases when you add parameters. An ARIMA(10,1,10) on BTC data will fit the training period beautifully but forecast the test period catastrophically — it memorised 20 parameters worth of noise. AIC/BIC and out-of-sample validation exist precisely to catch this overfitting.

③ Real World Link

Every financial model that “looked perfect in backtests” but failed in live trading suffered from overfitting to in-sample noise. Professional quant funds mandate walk-forward validation with hold-out test sets — training performance is a sanity check, not a success criterion. The test RMSE is the real scorecard.

M3.008 *Predict the Output* **EASY****Topic:** Holt-Winters — additive vs no trend

Without running the code, predict which fit will have lower AIC, and describe the shape of the forecasts.

```

1 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing, SimpleExpSmoothing
2 import numpy as np
3
4 np.random.seed(1)
5 t = np.arange(200)
6 data = 50 + 0.3 * t + np.random.randn(200)    # linear
   trend + noise
7
8 fit_ses = SimpleExpSmoothing(data).fit(optimized=True)
9 fit_hw = ExponentialSmoothing(data,
   trend="add").fit(optimized=True)
10
11 print(f"SES AIC : {fit_ses.aic:.2f}")
12 print(f"HW AIC : {fit_hw.aic:.2f}")
13 print(f"HW wins : {fit_hw.aic < fit_ses.aic}")
14
15 fc_ses = fit_ses.forecast(10)
16 fc_hw = fit_hw.forecast(10)
17 print(f"\nSES forecast (flat?): {fc_ses.iloc[0]:.2f} ...
   {fc_ses.iloc[-1]:.2f}")
18 print(f"HW forecast (trend?): {fc_hw.iloc[0]:.2f} ...
   {fc_hw.iloc[-1]:.2f}")

```

Predict: (i) which AIC is lower, (ii) shape of SES forecasts, (iii) shape of HW forecasts.

Solution**① Predicted Output**

```

SES AIC : [higher value, e.g. ~900+]
HW AIC : [lower value, e.g. ~750+]
HW wins : True

```

```

SES forecast (flat?): [~same value] ... [~same value --- FLAT
line]
HW forecast (trend?): [~start val] ... [higher by ~3 ---
UPWARD SLOPE]

```

```

1 # Reasoning:
2 # Data has genuine linear trend:  $y_t = 50 + 0.3t + \text{noise}$ 
3 # SES has no trend component -> misspecified -> HIGH AIC
4 # HW with trend="add" captures the 0.3/step trend -> LOW

```

```

AIC
5 #
6 # SES forecast shape: FLAT -- estimates level only, not
  slope
7 # All 10 forecasts ~= same value (weighted average of
  recent data)
8 # HW forecast shape: LINEAR UPWARD -- adds estimated trend
  each step
9 # Forecast at step h = level + h x estimated_slope
  (~0.3/step)
10 # -> forecasts increase by ~0.3 per step -> step 10 ~=
    step 1 + 3

```

② Plain English

When data has a genuine trend, adding the trend component (Holt's method) is statistically justified — the AIC improvement from capturing the trend structure far outweighs the 2-unit penalty for the extra parameter. SES forecasts look like a flat line, while Holt-Winters forecasts correctly extend the upward slope.

③ Common Mistake

Thinking SES and HW give the same forecast for the first step. They don't — SES's level estimate is a weighted average of all past data, while HW's level estimate accounts for the trend and adjusts accordingly. Even at step 1, HW's forecast is typically better for trending series.

④ Real World Link

BTC's long-run price history shows a strong upward trend. A SES model applied to BTC would produce a flat forecast (useless for trend-following), while Holt's method would project the upward trend forward. The BTC community's "stock-to-flow" models are essentially sophisticated trend extrapolations — Holt's method is the statistical version of that intuition.

M3.009 Fill in the Blank **EASY**

Topic: Computing MAPE and RMSE for forecast evaluation

Fill in the blanks to compute a proper forecast evaluation on a BTC hold-out set.

```

1 btc    = yf.download("BTC-USD", start="2019-01-01",
  end="2024-12-31")
2 close  = btc["Close"].dropna()
3
4 # Temporal split: last 90 days as test
5 train  = close.iloc[: _____] # everything except last 90
6 test   = close.iloc[_____: ]      # last 90 days
7

```

```

8 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
9 fit = ExponentialSmoothing(train,
   trend="add").fit(optimized=True)
10 fc = fit.forecast(_____) # match test length
11
12 errors = test._____ - fc.values
13 rmse = np._____(np.mean(errors _____ 2))
14 mape = np.mean(np.abs(errors / test.values)) * _____
15
16 print(f"Test RMSE : ${rmse:,.2f}")
17 print(f"Test MAPE : {mape:.2f}%")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
7
8 btc = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
9 close = btc["Close"].dropna()
10
11 # Temporal split: last 90 days as test
12 train = close.iloc[:-90] # negative indexing:
   everything except last 90
13 test = close.iloc[-90:] # last 90 days
14
15 fit = ExponentialSmoothing(train,
   trend="add").fit(optimized=True)
16 fc = fit.forecast(90) # forecast exactly 90 steps to
   match test length
17
18 errors = test.values - fc.values # .values converts to
   numpy for clean subtraction
19 rmse = np.sqrt(np.mean(errors ** 2)) # sqrt of mean
   squared error
20 mape = np.mean(np.abs(errors / test.values)) * 100 # x
   100 converts to percentage
21
22 print(f"Test RMSE : ${rmse:,.2f}")
23 print(f"Test MAPE : {mape:.2f}%")

```

② Plain English

Negative indexing (`iloc[-90:]`) elegantly selects the last 90 rows without needing to know the total length. The key discipline: forecast exactly as many steps as there are test observations. Always verify `len(test) == len(fc)` before computing metrics.

③ Common Mistake

Using `test - fc` directly without `.values` — if `test` is a pandas Series with a DatetimeIndex and `fc` is a Series with an integer index, pandas tries to align by index and produces all NaN. Converting both to numpy arrays with `.values` sidesteps index alignment issues entirely.

④ Real World Link

For BTC, a 90-day test RMSE of \$5,000–\$15,000 is typical — that’s 7–20% of price, which MAPE will confirm. This sounds large but is actually reasonable given BTC’s daily volatility of $\sim 3\%$. Any model claiming $\text{MAPE} < 2\%$ on BTC 90-day forecasts should be scrutinised very carefully for data leakage.

M3.010 Short Answer EASY

Topic: Durbin-Levinson algorithm purpose

In 2 sentences, explain what the Durbin-Levinson algorithm computes and why it is more efficient than direct matrix inversion for fitting AR models.

Solution

① Model Answer

The Durbin-Levinson algorithm recursively computes the partial autocorrelation function (PACF) and solves the Yule-Walker equations for AR model parameters — it builds the $\text{AR}(p)$ solution from the $\text{AR}(p-1)$ solution at each step, avoiding the need to solve a new linear system from scratch. This recursive structure reduces computation from $O(p^3)$ (direct matrix inversion of the $p \times p$ Toeplitz covariance matrix) to $O(p^2)$, making it significantly faster when estimating high-order AR models on long financial time series.

② Common Mistake

Thinking Durbin-Levinson is only relevant to theoretical statistics courses. It’s actually the algorithm running inside `statsmodels` every time you fit an AR model — understanding it explains why PACF values at each lag correspond to exactly the AR coefficients you’d get if you stopped the Durbin-Levinson recursion at that lag, which is why PACF is the right diagnostic for AR order selection.

③ Real World Link

When you call `plot_pacf(btc_returns, lags=40)`, the PACF values displayed are computed via the Durbin-Levinson algorithm. Each spike at lag k is the $\text{AR}(k)$ coefficient from a model that includes lags 1 through k — which is exactly why a significant PACF spike at lag 2 (and not at lag 3+) indicates that $\text{AR}(2)$ is the

appropriate model order.

Section B

Medium Problems — M3.011 to M3.022

M3.011 *Fill in the Blank* MEDIUM

Topic: Walk-forward validation (expanding window)

Fill in the blanks to implement expanding-window walk-forward validation for a Holt-Winters model on BTC.

```

1 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
2 import numpy as np
3
4 btc    = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
5 close  = btc["Close"].dropna().values
6
7 min_train = 252
8 errors    = []
9
10 for t in range(min_train, len(close) - _____):
11     train_window = close[_____ : t]           # expanding:
   start=0, end=t
12     actual_next  = close[_____]             # one step
   ahead actual
13
14     try:
15         fit      = ExponentialSmoothing(
16                     train_window, trend="add"
17                 ).fit(optimized=True, disp=False)
18         forecast = fit.forecast(_____) [0]
19         errors.append(actual_next - _____)
20     except Exception:
21         continue
22
23 errors = np.array(errors)
24 print(f"Walk-forward steps : {len(errors)}")
25 print(f"Walk-forward RMSE  :
   ${np.sqrt(np.mean(errors**2)):.2f}")
26 print(f"Walk-forward MAE   :
   ${np.mean(np.abs(errors)):.2f}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
7
8 btc    = yf.download("BTC-USD", start="2019-01-01",
9                     end="2024-12-31")
10 close = btc["Close"].dropna().values
11
12 min_train = 252
13 errors    = []
14
15 for t in range(min_train, len(close) - 1):  # stop at
16     len-1 so t always exists
17     train_window = close[0 : t]              # EXPANDING:
18     always starts at 0
19     actual_next  = close[t]                  # the value
20     we're forecasting
21
22     try:
23         fit      = ExponentialSmoothing(
24             train_window, trend="add"
25         ).fit(optimized=True, disp=False)
26         forecast = fit.forecast(1)[0]        #
27         1-step-ahead, take scalar
28         errors.append(actual_next - forecast)
29     except Exception:
30         continue
31
32 errors = np.array(errors)
33 print(f"Walk-forward steps : {len(errors)}")
34 print(f"Walk-forward RMSE  :
35       ${np.sqrt(np.mean(errors**2)):.2f}")
36 print(f"Walk-forward MAE   :
37       ${np.mean(np.abs(errors)):.2f}")

```

② Plain English

Walk-forward validation mimics real forecasting: at each step, you train on all available historical data and forecast one step ahead, then move forward one day and repeat. The expanding window grows over time (as a real model would see more data each day). The resulting error distribution is the most realistic estimate of how your model would perform in live deployment.

③ Common Mistake

Implementing a rolling (fixed-size) window instead of an expanding window — these are different. Expanding window = always start from day 0, grow end point. Rolling window = move both start and end point together. Expanding is more realistic for most financial applications since real systems accumulate history. Make sure `train_window = close[0:t]` not `close[t-252:t]`.

④ Real World Link

Every major quantitative hedge fund implements walk-forward validation before deploying any strategy. The walk-forward RMSE for BTC from a Holt-Winters model is the honest answer to “how accurate is this model?” — and it will be large, reflecting BTC’s true unpredictability.

M3.012 Find the Error MEDIUM

Topic: Holt-Winters — additive seasonal on growing series

This code fits a model and produces unreasonable forecasts (possibly going negative). Identify the specification error and fix it.

```

1 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
2
3 gold      = yf.download("GC=F", start="2019-01-01",
   end="2024-12-31")
4 monthly   = gold["Close"].dropna().resample("ME").last()
5
6 # Gold price tripled from ~$1300 to ~$2700 over this period
7 model = ExponentialSmoothing(
8     monthly,
9     trend="add",
10    seasonal="add",          # ERROR: wrong seasonal
                               type for this data
11    seasonal_periods=12
12)
13 fit      = model.fit(optimized=True)
14 forecast = fit.forecast(24)
15
16 print(f"Min forecast value: ${forecast.min():,.2f}") #
   may go negative
17 print(f"Max forecast value: ${forecast.max():,.2f}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf

```

```

3 import matplotlib.pyplot as plt
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
7
8 gold      = yf.download("GC=F", start="2019-01-01",
   end="2024-12-31")
9 monthly   = gold["Close"].dropna().resample("ME").last()
10
11 # FIX: Gold price more than doubled over 2019-2024
12 # Seasonal swings (e.g. Diwali demand) grow proportionally
   with price level
13 # -> This is MULTIPLICATIVE seasonality, not additive
14 # Additive seasonal model estimates fixed-size seasonal
   swings
15 # -> When applied to a series whose level doubled,
   additive swings
16 #     can become negative in the subtraction -> negative
   forecasts
17
18 model = ExponentialSmoothing(
19     monthly,
20     trend="add",
21     seasonal="mul",          # FIX: multiplicative
   for proportional swings
22     seasonal_periods=12
23 )
24 fit      = model.fit(optimized=True)
25 forecast = fit.forecast(24)
26
27 print(f"Min forecast value: ${forecast.min():,.2f}")    #
   positive now
28 print(f"Max forecast value: ${forecast.max():,.2f}")

```

② Plain English

When a time series grows substantially in level (Gold doubled from \$1300 to \$2700), seasonal swings also grow — a 3% Diwali bump is worth \$39 at \$1300 but \$81 at \$2700. Additive seasonal models assume fixed seasonal increments regardless of level — when the level is high and these fixed increments are subtracted in the seasonal decomposition, they can drive fitted values below zero.

③ Common Mistake

Defaulting to `seasonal="add"` without checking whether seasonal amplitude is constant or proportional. The diagnostic: plot the raw data and eyeball whether the seasonal peaks and troughs are roughly the same height throughout (→ additive) or grow/shrink with the series level (→ multiplicative). For price series that have trended significantly, multiplicative is almost always the right choice.

④ Real World Link

Additive seasonal models applied to strongly trending asset prices genuinely do produce negative forecasts in some configurations. Gold going from \$1300 to \$2700 while using an additive model calibrated on 2019 data would assign seasonal corrections appropriate for \$1300 prices to a \$2700 base, causing chaos in the seasonal adjustment.

M3.013 *Get the Order Right* MEDIUM

Topic: Walk-forward forecast evaluation workflow

These **8 steps** for proper walk-forward evaluation are scrambled. Write the correct order.

- (A) Compute error metrics (RMSE, MAE, MAPE) across all forecast steps
- (B) Set minimum training window size (e.g. 252 for 1 year of daily data)
- (C) Collect actual vs forecast pairs in a list
- (D) Download the time series data and clean it
- (E) For each time step t from min_train to end: fit model on data[0:t]
- (F) Plot actual vs forecasted values over the evaluation window
- (G) Forecast one step ahead and store the result
- (H) Print final evaluation report with metrics and model specification

Solution

① **Correct Order:** D → B → E → G → C → A → F → H

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
6
7 # STEP D: Download and clean data
8 btc = yf.download("BTC-USD", start="2021-01-01",
9   end="2024-12-31")
10 close = btc["Close"].dropna().values
11
12 # STEP B: Set minimum training window
13 min_train = 252
14 forecasts_list = []; actuals_list = []
15
16 # STEP E: Loop over time steps
17 for t in range(min_train, len(close) - 1):
18     train = close[0:t]
19     try:

```

```

20         fit = ExponentialSmoothing(train, trend="add").fit(
21             optimized=True, disp=False)
22         # STEP G: Forecast one step ahead
23         fc_val = fit.forecast(1)[0]
24         # STEP C: Store actual vs forecast pair
25         forecasts_list.append(fc_val)
26         actuals_list.append(close[t])
27     except Exception:
28         continue
29
30 actuals = np.array(actuals_list)
31 forecasts = np.array(forecasts_list)
32 errors = actuals - forecasts
33
34 # STEP A: Compute metrics
35 rmse = np.sqrt(np.mean(errors**2))
36 mae = np.mean(np.abs(errors))
37 mape = np.mean(np.abs(errors / actuals)) * 100
38
39 # STEP F: Plot
40 plt.figure(figsize=(14, 5))
41 plt.plot(actuals, label="Actual BTC",
42          color="steelblue", linewidth=0.8)
43 plt.plot(forecasts, label="HW Forecast", color="red",
44          linewidth=0.8, alpha=0.7)
45 plt.title("Walk-Forward Forecast Evaluation -- BTC
46           Holt-Winters")
47 plt.legend(); plt.show()
48
49 # STEP H: Final report
50 print(f"\n{'='*50}")
51 print(f" Walk-Forward Evaluation Report")
52 print(f" Model      : Holt-Winters (trend=add)")
53 print(f" Steps       : {len(errors)}")
54 print(f" RMSE        : ${rmse:,.2f}")
55 print(f" MAE         : ${mae:,.2f}")
56 print(f" MAPE        : {mape:,.2f}%")
57 print(f"{'='*50}")

```

② Plain English

The ordering is logical: data → configure → loop (fit → forecast → collect) → evaluate → report. The loop body must be E→G→C in that exact order: you can't forecast before fitting (G needs E) and you can't store a pair before forecasting (C needs G). Metrics (A) need all collected pairs, so A follows C outside the loop.

③ Common Mistake

Computing metrics inside the loop rather than after. If you print RMSE inside the loop, you get a new number at every step — none of which is the true evaluation metric. Metrics must be computed once, over the complete set of errors. Collect

inside the loop, evaluate outside.

④ Real World Link

This 8-step workflow is the industry-standard validation template for any financial time series model. Before approving a model for production use at a bank or fund, a model risk management team would require evidence of exactly this process — documented, reproducible, with all 8 steps present. Missing any step is grounds for rejection.

M3.014 Match the Following **MEDIUM**

Topic: Forecast error metrics — properties and use cases

Match each **metric** (Column A) to its **key property** (Column B).

#	Column A — Metric	#	Column B — Property
1	MAE	(a)	Scale-free — compare across assets of different price levels
2	MSE	(b)	Same units as original series; penalises large errors heavily
3	RMSE	(c)	Squares errors — large mistakes penalised disproportionately
4	MAPE	(d)	Average absolute error; robust to outliers; easy to interpret
5	AIC	(e)	Penalises model complexity; used for in-sample model selection

Solution

① Correct Matches

Metric	Match
1. MAE	(d) Average absolute error; robust to outliers; easy to interpret
2. MSE	(c) Squares errors — large mistakes penalised disproportionately
3. RMSE	(b) Same units as original series; penalises large errors heavily
4. MAPE	(a) Scale-free — compare across assets of different price levels
5. AIC	(e) Penalises complexity; used for in-sample model selection

② Plain English

Use RMSE when large errors are particularly costly (a \$10K BTC forecast miss is much worse than ten \$1K misses). Use MAE when you want a more robust metric that doesn't overweight occasional large errors. Use MAPE for dashboards comparing multiple assets — it's the only metric that means the same thing for a \$60K BTC and a 90K MRF share simultaneously.

③ Common Mistake

Using MAPE for return series. Returns are often close to zero — say 0.001 — and a forecast error of 0.002 gives $\text{MAPE} = 200\%$, which is technically correct but uninformative. MAPE is designed for series where the actual values are substantially non-zero. For returns, stick to MAE or RMSE.

④ Real World Link

In an equity research report comparing BTC, MRF, and Gold forecast models, MAPE is the correct metric for the headline comparison table — it puts all three on equal footing. RMSE in the model appendix is useful for readers who want absolute numbers. Both together give a complete picture.

M3.015 Multiple Choice **MEDIUM**

Topic: Holt-Winters damped trend

You fit two models to BTC data: `ExponentialSmoothing(close, trend="add")` and `ExponentialSmoothing(close, trend="add", damped_trend=True)`. Which statement is correct?

- (A) The damped model's forecasts grow faster because damping accelerates the trend
- (B) The damped model's forecasts converge to a constant as horizon increases; the non-damped model extrapolates the trend indefinitely
- (C) Both models produce identical long-horizon forecasts
- (D) Damping only affects the seasonal component, not the trend

Solution

① Correct Answer: (B)

② Plain English

The undamped Holt-Winters extrapolates the estimated trend indefinitely — if BTC was rising \$200/day at the end of training, it projects that exact growth rate forward forever. The damped model applies a dampening coefficient $\phi < 1$ that causes the trend to slow down and eventually flatten. For long horizons, damped forecasts are more realistic — trends rarely continue forever at the same rate.

③ Common Mistake

Thinking damping makes the model pessimistic. It doesn't make it pessimistic or optimistic — it just prevents the trend from running away to extreme values at long horizons. The damped model says “the trend will continue but probably not at this rate forever.” This is typically more honest than assuming exponential extrapolation.

④ Real World Link

BTC's undamped Holt-Winters 1-year forecast after the 2021 bull run would have predicted \$300,000+ BTC by 2023. The damped version would have given a more conservative estimate. BTC crashed to \$16,000 in 2022. This is why damped trend forecasts are preferred for volatile assets.

M3.016 *Write from Scratch* **MEDIUM****Topic:** Multi-model forecast comparison dashboard

Write code that: (1) downloads BTC data (2019–2024), (2) uses last 90 days as test set, (3) fits three models on training data: SES, Holt's (trend only), Holt-Winters (trend + damped), (4) generates 90-day forecasts from each, (5) plots all three forecasts against actuals, (6) prints a comparison table of RMSE, MAE, and MAPE for all three.

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import warnings
7 warnings.filterwarnings('ignore')
8 from statsmodels.tsa.holtwinters import
   SimpleExpSmoothing, ExponentialSmoothing
9
10 btc    = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
11 close = btc["Close"].dropna()
12
13 train = close.iloc[:-90]; test = close.iloc[-90:]; h =
   len(test)
14
15 fit_ses = SimpleExpSmoothing(train).fit(optimized=True)
16 fit_holt = ExponentialSmoothing(train, trend="add",
   damped_trend=False).fit(optimized=True)
17 fit_hw   = ExponentialSmoothing(train, trend="add",
   damped_trend=True).fit(optimized=True)
18
19
20
21 fc_ses   = fit_ses.forecast(h)
22 fc_holt  = fit_holt.forecast(h)
23 fc_hw    = fit_hw.forecast(h)
24
25 # Plot
26 fig, ax = plt.subplots(figsize=(16, 6))
27 ax.plot(train[-180:], color="steelblue", linewidth=1.2,
   label="Training Data")
28 ax.plot(test, color="black", linewidth=1.5,
   label="Actual (Test)")
29 ax.plot(test.index, fc_ses.values, color="orange",
   linewidth=2,
30         linestyle="--", label="SES Forecast")

```

```

31 ax.plot(test.index, fc_holt.values, color="red",
    linewidth=2,
32         linestyle="-. ", label="Holt's (Trend) Forecast")
33 ax.plot(test.index, fc_hw.values, color="green",
    linewidth=2,
34         label="HW Damped Forecast")
35 ax.axvline(x=train.index[-1], color="gray", linestyle=":",
    linewidth=1.5)
36 ax.set_title("BTC -- Multi-Model 90-Day Forecast
    Comparison")
37 ax.set_ylabel("USD"); ax.legend(fontsize=9);
    plt.tight_layout(); plt.show()
38
39 # Comparison table
40 def compute_metrics(actual, forecast):
41     e = actual.values - forecast.values
42     return np.sqrt(np.mean(e**2)), np.mean(np.abs(e)), \
43         np.mean(np.abs(e / actual.values)) * 100
44
45 rows = []
46 for name, fc in [("SES", fc_ses), ("Holt's Trend",
    fc_holt), ("HW Damped", fc_hw)]:
47     rmse, mae, mape = compute_metrics(test, fc)
48     rows.append({"Model": name, "RMSE ($)": f"{rmse:,.0f}",
49                 "MAE ($)": f"{mae:,.0f}", "MAPE (%)":
50                     f"{mape:.2f}"})
51
52 results_df = pd.DataFrame(rows)
53 print("\n" + "="*55)
54 print(" 90-Day Forecast Comparison -- BTC-USD")
55 print("="*55)
56 print(results_df.to_string(index=False))
57 print("="*55)

```

② Plain English

This script implements the complete model comparison workflow: split → fit multiple models → forecast → plot → evaluate. The comparison table makes winner selection objective. In practice, the “best” model changes depending on the test period — BTC’s regime changes (bull market vs bear market) can flip which smoother performs best.

③ Common Mistake

Comparing models on the training data instead of the test data — a model with lower training RMSE is not necessarily better at forecasting. Always plot both the training fit and the test forecast together so you can visually confirm the model isn’t overfitting.

④ Real World Link

This exact comparison — SES vs Holt vs Holt-Winters — is the standard benchmark analysis in any forecast model development project. Before deploying a sophisticated GARCH or LSTM model for BTC, you first establish these simple baselines. If your complex model can't beat Holt-Winters on MAPE, it's not worth the added complexity.

M3.017 Fill in the Blank **MEDIUM**

Topic: Estimation of mean, autocovariance, and autocorrelation

Fill in the blanks to manually compute the three key time series statistics for BTC returns and compare with `acf()`.

```

1 from statsmodels.tsa.stattools import acf
2
3 btc      = yf.download("BTC-USD", start="2019-01-01",
4                       end="2024-12-31")
5 returns = np.log(btc["Close"]).diff()._____()
6
7 n = len(returns)
8 mu_hat = returns._____()
9
10 def sample_autocovariance(series, k):
11     s = series.values
12     mu = s._____()
13     return np.sum([(s[t] - mu) * (s[t-k] - mu)
14                   for t in range(k, len(s))]) / _____
15
16 def sample_acf(series, k):
17     return sample_autocovariance(series, k) /
18         sample_autocovariance(series, _____)
19
20 print("Manual ACF:")
21 for k in [0, 1, 2, 3]:
22     print(f"    ACF(lag={k}) = {sample_acf(returns, k):.6f}")
23
24 acf_vals = acf(returns, nlags=_____, fft=True)
25 print("\nStatsmodels ACF:")
26 for k in [0, 1, 2, 3]:
27     print(f"    ACF(lag={k}) = {acf_vals[k]:.6f}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf

```

```

3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.stattools import acf
7
8 btc      = yf.download("BTC-USD", start="2019-01-01",
9                       end="2024-12-31")
10 returns = np.log(btc["Close"]).diff().dropna()
11 n       = len(returns)
12 mu_hat  = returns.mean()
13 print(f"Sample mean: {mu_hat:.8f}")
14
15 def sample_autocovariance(series, k):
16     s = series.values
17     mu = s.mean()
18     # Divide by n (biased) -- consistent with statsmodels
19     convention
20     return np.sum([(s[t] - mu) * (s[t-k] - mu)
21                   for t in range(k, len(s))]) / n
22
23 def sample_acf(series, k):
24     # ACF(k) = Cov(k) / Cov(0) -- normalise by lag-0 (=
25     variance)
26     return sample_autocovariance(series, k) /
27           sample_autocovariance(series, 0)
28
29 print("\nManual ACF:")
30 for k in [0, 1, 2, 3]:
31     print(f"    ACF(lag={k}) = {sample_acf(returns, k):.6f}")
32
33 acf_vals = acf(returns, nlags=3, fft=True) # nlags=3
34 gives lags 0,1,2,3
35 print("\nStatsmodels ACF:")
36 for k in [0, 1, 2, 3]:
37     print(f"    ACF(lag={k}) = {acf_vals[k]:.6f}")

```

② Plain English

Sample autocovariance at lag k measures how much the series correlates with itself shifted k periods. Dividing by the lag-0 autocovariance (which equals the variance) normalises it to the range $[-1, +1]$, giving the autocorrelation. The statsmodels `acf()` function computes exactly this, just faster using FFT for large datasets.

③ Common Mistake

Dividing by $n - k$ instead of n in the autocovariance denominator. Statsmodels uses n (biased estimator) for consistency across lags. Using $n - k$ gives an unbiased estimator but produces slight discrepancies with `acf()` output. Either is technically valid — just be consistent and match the convention of the library you're comparing against.

④ Real World Link

Understanding that `plot_acf()` is literally computing $[\sum (x_t - \mu)(x_{t-k} - \mu)] / [\sum (x_t - \mu)^2]$ demystifies the ACF plot completely. Every spike you see is a ratio of two dot products. This also explains why you need long time series for stable ACF estimates: with only 60 monthly observations, each ACF estimate has high sampling variance.

M3.018 Find the Error MEDIUM

Topic: AIC comparison across different datasets

Find the conceptual error in this model selection code.

```

1 from statsmodels.tsa.arima.model import ARIMA
2
3 gold    = yf.download("GC=F",      start="2019-01-01",
4                       end="2024-12-31")
5 monthly = gold["Close"].dropna().resample("ME").last()
6 btc     = yf.download("BTC-USD", start="2019-01-01",
7                       end="2024-12-31")
8 daily   = btc["Close"].dropna()
9
10
11 fit_gold = ARIMA(monthly, order=(1,1,1)).fit()
12 fit_btc  = ARIMA(daily,   order=(1,1,1)).fit()
13
14 # ERROR: Comparing AIC across models fitted on different
15 # datasets
16 print(f"Gold ARIMA(1,1,1) AIC: {fit_gold.aic:.2f}")
17 print(f"BTC  ARIMA(1,1,1) AIC: {fit_btc.aic:.2f}")
18
19 if fit_gold.aic < fit_btc.aic:
20     print("Gold ARIMA model is better!")      # WRONG
21     CONCLUSION
22 else:
23     print("BTC ARIMA model is better!")      # WRONG
24     CONCLUSION

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.arima.model import ARIMA
6
7 gold    = yf.download("GC=F",      start="2019-01-01",
8                       end="2024-12-31")

```

```

8 monthly = gold["Close"].dropna().resample("ME").last()
9 btc      = yf.download("BTC-USD", start="2019-01-01",
    end="2024-12-31")
10 daily    = btc["Close"].dropna()
11
12 fit_gold = ARIMA(monthly, order=(1,1,1)).fit()
13 fit_btc  = ARIMA(daily,   order=(1,1,1)).fit()
14
15 print(f"Gold ARIMA(1,1,1) AIC: {fit_gold.aic:.2f}
    [n={len(monthly)}]")
16 print(f"BTC ARIMA(1,1,1) AIC: {fit_btc.aic:.2f}
    [n={len(daily)}]")
17
18 # FIX: AIC is ONLY comparable across models fitted on
    IDENTICAL datasets
19 # AIC includes the log-likelihood which depends on:
20 # (a) the data values (Gold prices vs BTC prices --
    totally different scales)
21 # (b) sample size n (72 vs 1250 -- totally different)
22 # Comparing Gold AIC to BTC AIC is meaningless.
23
24 print("\n WRONG: AIC cannot compare models across
    different datasets/frequencies")
25 print(" CORRECT: compare models on THE SAME data:")
26
27 # Correct: compare ARIMA(1,1,1) vs ARIMA(2,1,0) on Gold
    monthly data
28 fit_gold_alt = ARIMA(monthly, order=(2,1,0)).fit()
29 print(f"\nGold ARIMA(1,1,1) AIC: {fit_gold.aic:.2f}")
30 print(f"Gold ARIMA(2,1,0) AIC: {fit_gold_alt.aic:.2f}")
31 winner = "(1,1,1)" if fit_gold.aic < fit_gold_alt.aic else
    "(2,1,0)"
32 print(f"Better Gold model: ARIMA{winner}")

```

② Plain English

AIC is a *relative* measure — it only ranks models competing to explain the same dataset. The AIC formula includes the log-likelihood term, which depends on both the data values and sample size. Gold monthly AIC and BTC daily AIC are computed from completely different data, so their numerical comparison is meaningless. AIC is like a judge's score in a cooking competition — you can compare scores within one competition, but not across different competitions with different dishes.

③ Common Mistake

This error is surprisingly common in student reports — comparing AIC values from a monthly model and a daily model, or from models trained on different assets. The rule is absolute: AIC comparisons are only valid when all candidate models are fitted on **exactly the same data** — same series, same frequency, same observations.

④ Real World Link

A legitimate AIC comparison: fit ARIMA(1,1,1), ARIMA(2,1,0), ARIMA(1,1,2) all on Gold monthly data and compare their AIC values. The winner is the best model for Gold monthly prices. Separately, do the same comparison for BTC daily data. Never cross-compare the results of these two competitions.

M3.019 Multiple Choice MEDIUM

Topic: FPE criterion — Final Prediction Error

The FPE (Final Prediction Error) criterion for AR(p) order selection is $FPE(p) = \hat{\sigma}^2 \cdot \frac{n+p+1}{n-p-1}$. How does FPE behave as p increases from 0 to $n/2$?

- (A) FPE always decreases as p increases — more parameters always means better prediction
- (B) FPE first decreases as the model captures genuine structure, then increases as the complexity penalty dominates
- (C) FPE is constant — it doesn't depend on p
- (D) FPE is equivalent to AIC and always selects the same order as AIC

Solution

① Correct Answer: (B)

② Plain English

FPE balances two competing forces: as p grows, $\hat{\sigma}^2$ (residual variance) decreases because the model captures more structure — this drives FPE down. But the fraction $(n+p+1)/(n-p-1)$ grows as p increases — this penalty drives FPE up. The optimal p is where these forces balance — typically at or near the true AR order. FPE is conceptually similar to AIC but derived from prediction error theory rather than information theory.

③ Common Mistake

Confusing FPE's behaviour with R^2 — R^2 always increases with more parameters (monotonically). FPE has a U-shaped behaviour: it finds a minimum. R^2 is useless for AR order selection; FPE (and AIC/BIC) correctly penalise complexity to find the U-curve minimum.

④ Real World Link

FPE was historically important before AIC became dominant. For BTC daily returns with $n = 1250$, the FPE fraction $(n+p+1)/(n-p-1)$ changes very slowly with p — so FPE and AIC will give nearly identical order selections. For short series like Gold monthly ($n = 72$), FPE's denominator changes faster with p , making it more sensitive to overfitting than AIC.

M3.020 Write from Scratch **MEDIUM****Topic:** Forecast evaluation with rolling window

Write code that implements a **rolling window** (fixed-size) walk-forward validation for Holt's method on Gold monthly prices, using a 36-month rolling window, and produces a formatted evaluation report.

Solution**① Correct Code**

```

1  !pip install yfinance statsmodels arch -q
2  import yfinance as yf
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import warnings
6  warnings.filterwarnings('ignore')
7  from statsmodels.tsa.holtwinters import
    ExponentialSmoothing
8
9  gold      = yf.download("GC=F", start="2016-01-01",
    end="2024-12-31")
10 monthly =
    gold["Close"].dropna().resample("ME").last().values
11 gold_dates =
    gold["Close"].dropna().resample("ME").last().index
12
13 window_size = 36      # 36 months = 3 years of training data
14
15 forecasts_list = []; actuals_list = []; dates_list = []
16
17 for t in range(window_size, len(monthly)):
18     # ROLLING window: always exactly 36 months, slides
    forward
19     train_window = monthly[t - window_size : t]
20     actual       = monthly[t]
21
22     try:
23         fit = ExponentialSmoothing(
24             train_window, trend="add",
25             damped_trend=True
26             ).fit(optimized=True, disp=False)
27         fc = fit.forecast(1)[0]
28         forecasts_list.append(fc);
29         actuals_list.append(actual)
30         dates_list.append(gold_dates[t] if t <
31             len(gold_dates) else t)
32     except Exception:
33         continue

```

```

32 actuals = np.array(actuals_list); forecasts =
    np.array(forecasts_list)
33 errors = actuals - forecasts
34
35 rmse = np.sqrt(np.mean(errors**2))
36 mae = np.mean(np.abs(errors))
37 mape = np.mean(np.abs(errors / actuals)) * 100
38 bias = np.mean(errors)
39
40 # Plot
41 fig, axes = plt.subplots(2, 1, figsize=(14, 8))
42 axes[0].plot(dates_list, actuals, color="steelblue",
    label="Actual Gold Price")
43 axes[0].plot(dates_list, forecasts, color="red",
    linestyle="--", label="1-Month Ahead
    Forecast")
45 axes[0].set_title("Gold Futures -- Rolling 36-Month Window
    Forecast")
46 axes[0].legend()
47
48 axes[1].bar(range(len(errors)), errors,
    color=["green" if e >= 0 else "red" for e in
    errors], alpha=0.7)
49
50 axes[1].axhline(0, color="black", linewidth=1)
51 axes[1].axhline(bias, color="orange", linewidth=2,
    linestyle="--", label=f"Mean Bias:
    ${bias:+.2f}")
52
53 axes[1].set_title("Forecast Errors Over Time");
    axes[1].legend()
54 plt.tight_layout(); plt.show()
55
56 print("="*55)
57 print(" Rolling Window Forecast Evaluation Report")
58 print(f" Asset : Gold Futures (GC=F)")
59 print(f" Window : {window_size} months (rolling, fixed
    size)")
60 print(f" Periods : {len(errors)} 1-step-ahead forecasts")
61 print("-"*55)
62 print(f" RMSE : ${rmse:,.2f}")
63 print(f" MAE : ${mae:,.2f}")
64 print(f" MAPE : {mape:.2f}%")
65 print(f" Bias : ${bias:+.2f} ({'under' if bias < 0
    else 'over'}-forecasting)")
66 print("="*55)

```

② Plain English

Rolling window validation trains on the most recent fixed 36 months only — it “forgets” older data by design. This is appropriate when you believe recent patterns are more informative than distant history — true for assets like Gold whose behaviour

changes with macro regimes. The bias metric is crucial: consistently negative bias means the model systematically under-predicts (Gold rising faster than the model expects).

③ Common Mistake

Confusing rolling window (fixed size, slides forward) with expanding window (always starts at $t = 0$). Expanding = “how well does the model do with all available history?”, rolling = “how well does the model do using only recent data?” For trend-following assets in changing regimes, rolling often wins; for stable relationships, expanding wins.

④ Real World Link

Gold’s regime change from a \$1200 bear market (2014–2018) to a \$2700 bull market (2020–2024) means a model trained on all data since 2014 carries stale bearish priors. A 36-month rolling window adapts to the current regime — this is exactly the reasoning behind rolling-window models in professional risk management systems.

M3.021 *Fill in the Blank* MEDIUM

Topic: Residual diagnostic — autocorrelation in errors

Fill in the blanks to check whether Holt-Winters residuals for BTC have autocorrelation structure.

```

1 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
2 from statsmodels.stats.diagnostic import _____
3 from statsmodels.graphics.tsaplots import plot_acf
4
5 btc = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
6 close = btc["Close"].dropna()
7
8 fit = ExponentialSmoothing(close, trend="add",
   damped_trend=True).fit(optimized=True)
9
10 # Extract residuals
11 resid = fit._____ - fit._____
12 resid = resid._____()
13
14 # Plot ACF of residuals
15 fig, ax = plt.subplots(figsize=(12, 4))
16 _____(resid, lags=30, alpha=0.05, ax=ax, zero=False)
17 ax.set_title("ACF of HW Residuals -- want all inside blue
   bands")
18 plt.show()
19
20 # Ljung-Box formal test
21 lb = acorr_ljungbox(_____, lags=20, return_df=True)
22 print(f"Residual Ljung-Box min p-value:
   {lb['lb_pvalue'].min():.4f}")

```

```

23
24 if lb["lb_pvalue"].min() > 0.05:
25     print("Residuals are white noise -- HW model adequate")
26 else:
27     print("Residuals have autocorrelation -- try ARIMA or
        SARIMA instead")

```

Solution

① Correct Code

```

1  !pip install yfinance statsmodels arch -q
2  import yfinance as yf
3  import matplotlib.pyplot as plt
4  import warnings
5  warnings.filterwarnings('ignore')
6  from statsmodels.tsa.holtwinters import
    ExponentialSmoothing
7  from statsmodels.stats.diagnostic import acorr_ljungbox
8  from statsmodels.graphics.tsaplots import plot_acf
9
10 btc = yf.download("BTC-USD", start="2019-01-01",
    end="2024-12-31")
11 close = btc["Close"].dropna()
12
13 fit = ExponentialSmoothing(
14     close, trend="add", damped_trend=True
15     ).fit(optimized=True)
16
17 # Residuals = actual minus fitted values
18 resid = close - fit.fittedvalues # fittedvalues gives
    in-sample fitted values
19 resid = resid.dropna() # first few may be NaN
    due to initialisation
20
21 fig, ax = plt.subplots(figsize=(12, 4))
22 plot_acf(resid, lags=30, alpha=0.05, ax=ax, zero=False)
23 ax.set_title("ACF of HW Residuals -- want all inside blue
    bands")
24 plt.show()
25
26 lb = acorr_ljungbox(resid, lags=20, return_df=True)
27 print(f"Residual Ljung-Box min p-value:
    {lb['lb_pvalue'].min():.4f}")
28
29 if lb["lb_pvalue"].min() > 0.05:
30     print("Residuals are white noise -- HW model adequate
        for mean dynamics")

```

```

31 else:
32     print("Residuals have autocorrelation -- consider
        ARIMA or seasonal model")

```

② Plain English

Residual diagnostics are the final exam any forecasting model must pass — not just ARIMA models. A well-specified Holt-Winters model should leave behind white noise residuals. Significant autocorrelation in residuals means the exponential smoothing didn't capture some repeating structure — perhaps a seasonal pattern you didn't model.

③ Common Mistake

Computing residuals as `fit.resid` for Holt-Winters models — this attribute exists in ARIMA fits but the equivalent for `ExponentialSmoothing` is computed as `observed - fittedvalues`. Always check the statsmodels documentation for the specific model class you're using; attribute names differ between model families.

④ Real World Link

For BTC's Holt-Winters model, residuals will typically show volatility clustering (large residuals followed by large residuals) even if there's no mean autocorrelation. The ACF of squared residuals (not raw residuals) would show significant spikes — the fingerprint of ARCH/GARCH effects. This is the diagnostic that motivates the GARCH models in Module 4.

M3.022 Find the Error MEDIUM

Topic: Naïve benchmark — wrong baseline

This code claims to build a “naïve benchmark” but implements it incorrectly. Find and fix it.

```

1  import numpy as np
2
3  btc    = yf.download("BTC-USD", start="2019-01-01",
4                      end="2024-12-31")
5  close  = btc["Close"].dropna().values
6
7  train  = close[:-90]; test  = close[-90:]
8
9  # WRONG naive forecast -- uses actual test values as
10     inputs!
11  naive_forecast = np.array([test[t] for t in
12                             range(len(test) - 1)] + [test[-1]])
13
14  errors = test - naive_forecast
15  rmse   = np.sqrt(np.mean(errors**2))
16  print(f"Naive RMSE: ${rmse:,.2f}")    # unrealistically
17                                         small

```

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 btc = yf.download("BTC-USD", start="2019-01-01",
8   end="2024-12-31")
9 close = btc["Close"].dropna().values
10
11 train = close[:-90]; test = close[-90:]
12
13 # FIX: Two standard naive benchmarks -- NO peeking at test
14 data:
15
16 # NAIVE 1: Random Walk -- forecast is always the last
17 TRAINING value
18 naive_rw = np.full(len(test), train[-1])
19
20 # NAIVE 2: One-step naive -- use previous actual as
21 forecast (true random walk)
22 # forecast[t] = actual[t-1], seeded from last training
23 point
24 naive_os = np.concatenate([[train[-1]], test[:-1]])
25
26 rmse_rw = np.sqrt(np.mean((test - naive_rw)**2))
27 rmse_os = np.sqrt(np.mean((test - naive_os)**2))
28
29 print("Naive Benchmarks:")
30 print(f" Random Walk (constant) RMSE: ${rmse_rw:,.2f}")
31 print(f" One-step random walk RMSE: ${rmse_os:,.2f}")
32 print(f"\nAny forecast model must beat BOTH to be
33   considered useful.")

```

② Plain English

The original code used `test[t]` — the actual future value — as the “forecast,” which is pure cheating. A proper naïve benchmark uses only information available at forecast time. The constant random walk (`train[-1]` for all periods) says “price stays at today’s level forever.” The one-step naïve uses yesterday’s actual as today’s forecast — valid because at each step you’d have yesterday’s actual available.

③ Common Mistake

Not including a naïve benchmark at all — then claiming a model with MAPE=8% is “good” without knowing whether the naïve baseline achieves MAPE=5%. A model that doesn’t beat a naïve benchmark is not useful. The naïve random walk is the minimum bar every model must clear before being considered for production use.

④ Real World Link

The random walk is one of the hardest benchmarks to beat for liquid asset prices. Eugene Fama's Efficient Market Hypothesis essentially states that the naïve random walk is the theoretically optimal forecast for equity prices. Any forecasting model for BTC, MRF, or Gold should be benchmarked against the random walk first — if it can't beat it, it's not generating alpha.

Section C

Hard Problems — M3.023 to M3.034

M3.023 *Fill in the Blank* **HARD**

Topic: Schwarz BIC derivation and sensitivity analysis

Fill in the blanks to implement BIC from scratch and compare it to statsmodels output across multiple models.

```

1 from statsmodels.tsa.arima.model import ARIMA
2
3 btc      = yf.download("BTC-USD", start="2019-01-01",
4                       end="2024-12-31")
5 returns  = np.log(btc["Close"]).diff().dropna()
6 n        = len(returns)
7
8 def manual_bic(log_likelihood, k, n):
9     # BIC = k*log(n) - 2*log(L)
10    return _____ * np.log(_____) - 2 * _____
11
12 def manual_aic(log_likelihood, k):
13     # AIC = 2k - 2*log(L)
14    return 2 * _____ - 2 * _____
15
16 orders = [(1,0,0), (1,0,1), (2,0,2)]
17 print(f{'Model':15} {'k':>4} {'LogL':>10} {'AIC_M':>10}
18       {'AIC_S':>10} {'BIC_M':>10} {'BIC_S':>10})
19 print("-"*75)
20
21 for order in orders:
22     fit = ARIMA(returns, order=order).fit()
23     k    = fit._____ + 1
24     logL = fit._____
25     print(f"ARIMA{str(order):10} {k:>4} {logL:>10.2f} "
26           f"{manual_aic(logL,k):>10.2f} {fit.aic:>10.2f} "
27           f"{manual_bic(logL,k,n):>10.2f} "
28           f"{fit.bic:>10.2f}")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import warnings
5 warnings.filterwarnings('ignore')
6 from statsmodels.tsa.arima.model import ARIMA
7
8 btc      = yf.download("BTC-USD", start="2019-01-01",
9                       end="2024-12-31")
10 returns = np.log(btc["Close"]).diff().dropna()
11 n        = len(returns)
12
13 def manual_bic(log_likelihood, k, n):
14     # BIC = k*log(n) - 2*log(L)
15     return k * np.log(n) - 2 * log_likelihood
16
17 def manual_aic(log_likelihood, k):
18     # AIC = 2k - 2*log(L)
19     return 2 * k - 2 * log_likelihood
20
21 orders = [(1,0,0), (1,0,1), (2,0,2)]
22 print(f"{'Model':15} {'k':>4} {'LogL':>10} "
23       f"{'AIC_M':>10} {'AIC_S':>10} {'BIC_M':>10} "
24       f"{'BIC_S':>10}")
25 print("-"*75)
26
27 for order in orders:
28     fit = ARIMA(returns, order=order).fit()
29     k    = fit.df_model + 1 # df_model = AR+MA params;
30         +1 for intercept/variance
31     logL = fit.llf          # llf = log-likelihood
32
33     aic_m = manual_aic(logL, k)
34     bic_m = manual_bic(logL, k, n)
35     match_a = "v" if abs(aic_m - fit.aic) < 0.5 else "x"
36     match_b = "v" if abs(bic_m - fit.bic) < 0.5 else "x"
37
38     print(f"ARIMA{str(order):10} {k:>4} {logL:>10.2f} "
39           f"{aic_m:>10.2f} {fit.aic:>10.2f}{match_a} "
40           f"{bic_m:>10.2f} {fit.bic:>10.2f}{match_b}")
41
42 print(f"\nSample size n = {n}")
43 print(f"log(n)           = {np.log(n):.4f}")
44 print(f"BIC/AIC penalty ratio: {np.log(n)/2:.2f}x per
45       parameter")

```

② Plain English

Implementing AIC and BIC from scratch confirms that these are not mysterious black-box numbers — they're simple functions of log-likelihood (reward for fit) and parameter count (penalty for complexity). For BTC daily data with $n \approx 1250$, $\log(n) \approx 7.1$, so BIC's penalty is $7.1/2 \approx 3.55$ times AIC's. This is why BIC almost always selects simpler models than AIC on large datasets.

③ Common Mistake

Confusion about what k counts. In statsmodels ARIMA, `df_model` counts the AR and MA parameters but may or may not include the constant and error variance — check the summary output's "No. of Parameters" field and cross-reference with your manual calculation. Small discrepancies (< 1 unit in AIC) are usually due to constant term counting conventions.

④ Real World Link

When a fund manager asks "which model did you use and why?" — being able to say "ARIMA(1,0,1) because it minimised $\text{BIC} = k \log(1250) - 2 \log L$, outperforming ARIMA(2,0,2) by 8.3 BIC units" demonstrates genuine understanding rather than just running `auto_arima()` and accepting defaults.

M3.024 Find the Error **HARD**

Topic: Forecast interval coverage — wrong confidence level

This code claims to produce 95% confidence intervals but actually produces something narrower. Find the **three errors** in the CI construction.

```

1 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
2 from scipy.stats import norm
3
4 btc = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
5 close = btc["Close"].dropna()
6
7 fit = ExponentialSmoothing(close, trend="add").fit()
8 fc_mean = fit.forecast(30)
9 resid_std = fit.resid.std() # ERROR 1: HW has no .resid
   attribute
10
11 z = norm.ppf(0.95) # ERROR 2: 0.95 gives
   one-sided, not two-sided 95%
12 std_h = np.array([resid_std * h # ERROR 3: std grows as
   sqrt(h), not linearly
13                     for h in range(1, 31)])
14
15 lower = fc_mean.values - z * std_h
16 upper = fc_mean.values + z * std_h
17
18 print(f"CI width at step 1 : ${upper[0]-lower[0]:,.2f}")

```

```
19 print(f"CI width at step 30: ${upper[29]-lower[29]:,.2f}")
```

Solution

① Correct Code

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import warnings
6 warnings.filterwarnings('ignore')
7 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
8 from scipy.stats import norm
9
10 btc    = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
11 close  = btc["Close"].dropna()
12
13 fit = ExponentialSmoothing(close,
   trend="add").fit(optimized=True)
14
15 # FIX 1: HW residuals = observed minus fitted values (no
   .resid attribute)
16 resid    = (close - fit.fittedvalues).dropna()
17 resid_std = resid.std()
18
19 # FIX 2: For two-sided 95% CI, use z = norm.ppf(0.975) not
   norm.ppf(0.95)
20 # norm.ppf(0.95) = 1.645 -> 90% two-sided CI
21 # norm.ppf(0.975) = 1.96 -> 95% two-sided CI (CORRECT)
22 z = norm.ppf(0.975)
23
24 # FIX 3: Forecast uncertainty grows as sqrt(h), not
   linearly with h
25 # Var(h-step) = h x sigma^2 -> std(h) = sigma x sqrt(h)
26 h_values  = np.arange(1, 31)
27 std_h     = resid_std * np.sqrt(h_values)  # CORRECT:
   grows as sqrt(h)
28
29 fc_mean   = fit.forecast(30)
30 lower     = fc_mean.values - z * std_h
31 upper     = fc_mean.values + z * std_h
32
33 print(f"z-value (correct for 95% two-sided): {z:.4f}")
34 print(f"CI width at step 1 : ${upper[0]-lower[0]:,.2f}")
35 print(f"CI width at step 30: ${upper[29]-lower[29]:,.2f}")
```

```

36 print(f"Width growth factor:
    {(upper[29]-lower[29])/(upper[0]-lower[0]):.2f}x")
37 print(f"Expected (sqrt(30)/sqrt(1) = {np.sqrt(30):.2f}x)")

```

② Plain English

Three distinct errors: (1) Wrong attribute name — HW uses `fittedvalues` not `.resid`; (2) Wrong quantile — for two-sided 95% CI you need the 97.5th percentile ($z = 1.96$), not the 95th ($z = 1.645$, which gives 90% two-sided); (3) Wrong uncertainty growth model — forecast standard error grows as $\sqrt{h}$ (random walk property), not linearly with h . If std grew linearly, a 30-day CI would be $30\times$ wider than a 1-day CI; with $\sqrt{h}$ scaling it's only $\sqrt{30} \approx 5.5\times$ wider.

③ Common Mistake

Error 2 (wrong quantile) is by far the most common CI mistake. Students memorise “95% $\rightarrow$ 1.96” but forget that 1.96 corresponds to `norm.ppf(0.975)` — you look up the 97.5th percentile to get a two-sided 95% interval. If you use `norm.ppf(0.95)` = 1.645, your intervals are actually 90% intervals, not 95%.

④ Real World Link

The $\sqrt{h}$ uncertainty growth is fundamental to options pricing — a 30-day option has $\sqrt{30}/\sqrt{1} \approx 5.5\times$ more uncertainty than a 1-day option, which is exactly why long-dated options are more expensive. This square-root-of-time rule for volatility scaling is used by every risk manager in financial markets, including in BTC crypto derivatives.

M3.025 Multiple Choice **HARD**

Topic: Model selection with structural break

You fit ARIMA(1,1,1) to MRF daily data for 2019–2024. The model has strong in-sample fit (low AIC) but poor out-of-sample performance after March 2020 (COVID crash). Which is the most likely explanation and best remedy?

- (A) The ARIMA order is wrong — try ARIMA(2,1,2) to improve out-of-sample performance
- (B) The model captured pre-COVID dynamics and a structural break in March 2020 changed the data-generating process — train on post-2020 data only or use a rolling window
- (C) The AIC is not a reliable criterion — switch to BIC for better out-of-sample performance
- (D) Out-of-sample performance is always worse than in-sample — this is normal and no action is needed

Solution**① Correct Answer: (B)****② Plain English**

A structural break is a permanent change in the data-generating process — the statistical properties (mean, variance, autocorrelation) shift and don't revert. Using data from both regimes to estimate a single model gives you parameters that represent a weighted average of two different processes — meaningful for neither. The correct remedy is either training on post-break data only, or using a rolling window that naturally weights recent data more.

③ Common Mistake

Choosing (A) — adding more AR/MA parameters does not fix a structural break. A perfectly specified ARIMA(10,1,10) trained on 2019–2024 data will still fail if 2019 and 2024 have fundamentally different dynamics. The problem is temporal heterogeneity, not model order misspecification. More parameters won't solve a regime change problem.

④ Real World Link

COVID-19 was a textbook structural break for all three course datasets. Any model trained on 2019 data and deployed in March 2020 without adaptation would have catastrophically misfired. This is why financial models are retrained monthly or quarterly in professional settings.

M3.026 Write from Scratch HARD

Topic: Diebold-Mariano test for forecast comparison

Write code implementing the **Diebold-Mariano (DM) test** to formally test whether two models produce statistically different forecast accuracy, applied to BTC 1-step-ahead forecasts from SES vs Holt's method (walk-forward).

Solution**① Correct Code**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import scipy.stats as stats
5 import matplotlib.pyplot as plt
6 import warnings
7 warnings.filterwarnings('ignore')
8 from statsmodels.tsa.holtwinters import
   SimpleExpSmoothing, ExponentialSmoothing
9
10 btc    = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
11 close = btc["Close"].dropna().values

```

```

12
13 min_train = 252
14 fc_ses_all = []; fc_hw_all = []; actual_all = []
15
16 for t in range(min_train, len(close) - 1):
17     train = close[0:t]
18     actual_all.append(close[t])
19     try:
20         ses =
21             SimpleExpSmoothing(train).fit(optimized=True,
22                                           disp=False)
23         fc_ses_all.append(ses.forecast(1)[0])
24     except:
25         fc_ses_all.append(np.nan)
26     try:
27         hw = ExponentialSmoothing(train, trend="add",
28                                   damped_trend=True).fit(optimized=True,
29                                                           disp=False)
30         fc_hw_all.append(hw.forecast(1)[0])
31     except:
32         fc_hw_all.append(np.nan)
33
34 actuals = np.array(actual_all)
35 fc_ses = np.array(fc_ses_all)
36 fc_hw = np.array(fc_hw_all)
37
38 # Remove NaN rows
39 mask = ~(np.isnan(fc_ses) | np.isnan(fc_hw))
40 actuals = actuals[mask]; fc_ses = fc_ses[mask]; fc_hw =
41     fc_hw[mask]
42
43 # Diebold-Mariano Test
44 e1 = (actuals - fc_ses)**2      # SES squared errors
45 e2 = (actuals - fc_hw)**2      # HW squared errors
46 d = e1 - e2                    # positive d_t -> SES worse
47     than HW
48
49 n = len(d)
50 d_bar = np.mean(d)
51 se_dm = np.sqrt(np.var(d, ddof=0) / n)    # simple SE for
52     h=1 step ahead
53 dm_stat = d_bar / se_dm
54 p_value = 2 * (1 - stats.t.cdf(abs(dm_stat), df=n-1))
55
56 ses_rmse = np.sqrt(np.mean(e1)); hw_rmse =
57     np.sqrt(np.mean(e2))
58
59 print("="*55)
60 print("    Diebold-Mariano Test: SES vs HW Damped")

```

```

54 print("  Dataset: BTC-USD, 1-step-ahead walk-forward")
55 print("="*55)
56 print(f"  Steps evaluated   : {n}")
57 print(f"  SES RMSE             : ${ses_rmse:,.2f}")
58 print(f"  HW RMSE              : ${hw_rmse:,.2f}")
59 print(f"  Mean loss diff       : ${d_bar:,.2f} (positive ->
    SES worse)")
60 print(f"  DM Statistic         : {dm_stat:.4f}")
61 print(f"  p-value (2-sided): {p_value:.4f}")
62 print("="*55)
63 if p_value < 0.05:
64     winner = "HW Damped" if d_bar > 0 else "SES"
65     print(f"  REJECT H0: Accuracy SIGNIFICANTLY DIFFERENT")
66     print(f"  -> {winner} is statistically superior at 5%
        level")
67 else:
68     print(f"  FAIL to reject H0: No significant accuracy
        difference")
69     print(f"  -> Choose simpler model (SES) on parsimony
        grounds")
70 print("="*55)

```

② Plain English

The DM test answers “is Model A significantly better than Model B, or could the observed accuracy difference just be sampling noise?” H_0 says both models have equal forecast accuracy. If $p < 0.05$, we conclude the models are genuinely different — one is statistically better. RMSE difference alone isn’t enough without a significance test.

③ Common Mistake

Concluding “HW is better because RMSE is lower” without checking statistical significance. A difference of \$100 RMSE might be statistically insignificant if the loss differential time series is very noisy. The DM test properly accounts for the sampling variability of forecast errors. In published research, you must report DM test results, not just RMSE comparisons.

④ Real World Link

The DM test (Diebold & Mariano, 1995) is mandatory in academic forecast comparison papers. If you submit a paper claiming “our LSTM beats ARIMA for BTC forecasting” without a DM test, reviewers will reject it. At a quant fund, the analytics team runs DM tests before declaring that a new model is better than the incumbent and worth the cost of deployment.

M3.027 Predict the Output **HARD**

Topic: Holt-Winters — optimal smoothing parameters

Without running this code, predict the sign and approximate magnitude of each smoothing parameter and the forecast shape.

```

1 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
2 import numpy as np, pandas as pd
3
4 np.random.seed(5)
5 n = 300; t = np.arange(n)
6 trend = 0.1 * t
7 seasonal = 20 * np.sin(2 * np.pi * t / 12)
8 noise = np.random.randn(n) * 2
9 y = pd.Series(50 + trend + seasonal + noise)
10
11 fit = ExponentialSmoothing(y, trend="add", seasonal="add",
12                             seasonal_periods=12).fit(optimized=True)
13
14 print(f"Alpha (level)      :
15       {fit.params['smoothing_level']:.4f}")
16 print(f"Beta (trend)       :
17       {fit.params['smoothing_trend']:.4f}")
18 print(f"Gamma (seasonal)   :
19       {fit.params['smoothing_seasonal']:.4f}")
20 fc = fit.forecast(24)
21 print(f"Forecast step 1   : {fc.iloc[0]:.2f}")
22 print(f"Forecast step 24  : {fc.iloc[-1]:.2f}")
23 print(f"Forecast rising?  : {fc.iloc[-1] > fc.iloc[0]}")

```

Predict: direction of each parameter (high/low), and whether the 24-step forecast rises.

Solution

① Predicted Output

```

Alpha (level) : [low, approximately 0.1--0.4]
Beta (trend)  : [very low, approximately 0.01--0.1]
Gamma (seasonal) : [moderate to high, approximately 0.3--0.8]
Forecast step 1 : [approximately 80--100]
Forecast step 24 : [approximately forecast_1 + 2.4]
Forecast rising? : True

```

```

1 # Reasoning:
2 # ALPHA (level): Strong structure (trend+seasonal) -> Low
   alpha preferred
3 # Level should be updated slowly: alpha ~ 0.1-0.3
4 #
5 # BETA (trend): Trend is very smooth (perfectly linear:
   0.1*t)
6 # Very low beta: trend estimate barely needs updating:
   beta ~ 0.01-0.05

```

```

7 #
8 # GAMMA (seasonal): Strong seasonality with amplitude 20
9 #   Moderate-to-high gamma: seasonal factors update
   responsively: gamma ~ = 0.3-0.7
10 #
11 # Forecast shape:
12 #   Trend component ~ = 0.1/step x 24 steps = +2.4 over 24
   steps -> True
13 #   Seasonal component oscillates around the rising trend
   line

```

② Plain English

The smoothing parameters are chosen by the optimizer to minimise in-sample squared error. When the data has a very clean linear trend, β (trend smoothing) approaches zero because the trend estimate from early in the series is already very accurate and barely needs updating. Strong seasonality with clean cycles pushes γ higher because each new cycle confirms the seasonal pattern.

③ Common Mistake

Expecting high alpha because the data is “moving” — alpha doesn’t measure how much the series changes, it measures how much weight to give to the most recent observation. With 300 points and clear structure, the optimizer finds that putting heavy weight on recent observations (high alpha) is unnecessary.

④ Real World Link

For Gold monthly data with a clear upward trend since 2019 and annual seasonal patterns, you’d expect similar parameter values: low β (the trend is persistent), moderate γ (seasonal factors are stable but not perfectly fixed), and low-to-moderate α . Checking these parameter values after fitting is a useful sanity check — implausibly extreme values ($\alpha > 0.95$ or $\beta > 0.5$) suggest model misspecification.

M3.028 Fill in the Blank **HARD**

Topic: Chow test for structural break detection

Fill in the blanks to implement a Chow test for a structural break at the COVID crash (March 2020) in MRF returns.

```

1 import scipy.stats as stats
2 from statsmodels.tsa.ar_model import AutoReg
3
4 mrf      = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
5 returns = np.log(mrf["Close"]).diff()._____()
6
7 break_date = "2020-04-01"

```

```

8 pre_covid = returns[: _____]
9 post_covid = returns[_____:]
10
11 def ar1_sse(series):
12     fit = AutoReg(series, lags=1).fit()
13     sse = np.sum(fit._____ ** 2)
14     n = len(_____)
15     return sse, n
16
17 sse_pre, n_pre = ar1_sse(pre_covid)
18 sse_post, n_post = ar1_sse(post_covid)
19
20 fit_full = AutoReg(returns, lags=1).fit()
21 sse_full = np.sum(fit_full.resid ** 2)
22 n_full = len(returns)
23
24 k = 2 # AR(1): intercept + AR(1) coefficient
25
26 chow_num = (sse_full - sse_pre - sse_post) / _____
27 chow_den = (sse_pre + sse_post) / (n_full - 2 * _____)
28 chow_f = chow_num / _____
29
30 p_chow = 1 - stats.f.cdf(chow_f, dfn=_____, dfd=n_full
31 - 2*k)
32
33 print(f"Chow F-stat : {chow_f:.4f}")
34 print(f"p-value : {p_chow:.4f}")
35 if p_chow < 0.05:
36     print("Structural break detected at 5% significance")
37 else:
38     print("No significant structural break detected")

```

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import scipy.stats as stats
5 import warnings
6 warnings.filterwarnings('ignore')
7 from statsmodels.tsa.ar_model import AutoReg
8
9 mrf = yf.download("MRF.NS", start="2019-01-01",
10 end="2024-12-31")
11 returns = np.log(mrf["Close"]).diff().dropna()

```

```

12 break_date = "2020-04-01"
13 pre_covid = returns[:break_date]
14 post_covid = returns[break_date:]
15
16 def ar1_sse(series):
17     fit = AutoReg(series, lags=1).fit()
18     sse = np.sum(fit.resid ** 2)    # sum of squared
    residuals
19     n = len(series)
20     return sse, n
21
22 sse_pre, n_pre = ar1_sse(pre_covid)
23 sse_post, n_post = ar1_sse(post_covid)
24
25 fit_full = AutoReg(returns, lags=1).fit()
26 sse_full = np.sum(fit_full.resid ** 2)
27 n_full = len(returns)
28
29 k = 2    # AR(1) has 2 parameters
30
31 # Chow F-statistic
32 chow_num = (sse_full - sse_pre - sse_post) / k
33 chow_den = (sse_pre + sse_post) / (n_full - 2 * k)
34 chow_f = chow_num / chow_den
35
36 # p-value: F(k, n-2k) distribution
37 p_chow = 1 - stats.f.cdf(chow_f, dfn=k, dfd=n_full - 2*k)
38
39 print(f"\nChow Test for Structural Break -- MRF Returns")
40 print(f"Break point : {break_date}")
41 print(f"Pre-COVID n : {n_pre} | SSE: {sse_pre:.6f}")
42 print(f"Post-COVID n : {n_post} | SSE: {sse_post:.6f}")
43 print(f"Chow F-stat : {chow_f:.4f}")
44 print(f"p-value : {p_chow:.4f}")
45 if p_chow < 0.05:
46     print("Structural break detected -- pre- and
    post-COVID dynamics differ")
47     print("Recommendation: use post-2020 data only for
    current model training")
48 else:
49     print("No significant structural break at this date")

```

② Plain English

The Chow test asks: “Is a single model fitted to the whole period significantly worse than two separate models for each sub-period?” If the full-period residuals (SSE_{full}) are much larger than the sum of sub-period residuals (SSE_{pre} + SSE_{post}), the F-statistic is large and p-value is small, confirming that the data-generating process changed at the break point.

③ Common Mistake

Choosing the break point after looking at the data (“I’ll test where the biggest break seems to be”). This data-snooping inflates the Type I error rate. Break points should be specified a priori based on economic reasoning (COVID crash date was known to everyone) or detected using sequential tests that adjust for multiple testing.

④ Real World Link

Structural break testing is standard in financial econometrics before presenting any time series model. The Chow test provides the statistical evidence to justify splitting the sample or using a rolling window — turning what might look like cherry-picking into a defensible, quantified decision.

M3.029 Short Answer HARD

Topic: In-sample vs out-of-sample — overfitting mechanism

In 2 sentences, explain the exact mathematical mechanism by which a higher-order ARIMA model overfits in-sample while underperforming out-of-sample, using AIC as the reference framework.

Solution**① Model Answer**

When you add unnecessary parameters to an ARIMA model, the optimizer adjusts those parameters to fit the specific noise realisations in the training data (reducing training RSS and improving log-likelihood), but these noise-fitted parameters will not generalise — when new data arrives with different noise, the extra parameters add bias rather than reduce variance, causing test RMSE to rise. AIC attempts to correct for this by adding a penalty of +2 per parameter, but this correction is approximate — models that overfit on small samples can still achieve better AIC while worsening out-of-sample performance, which is why direct walk-forward evaluation is always the final arbiter.

② Common Mistake

Saying “overfitting means the model is too complex.” Overfitting is specifically about the model learning noise that doesn’t repeat. An ARIMA(5,1,5) might legitimately need all 10 parameters if the returns series has genuine high-order ARMA structure. The problem is when those 10 parameters are capturing unrepeatable noise — and the only way to know which is happening is out-of-sample validation.

③ Real World Link

The classic cautionary tale: fitting a complex model to BTC’s 2019–2021 bull market data perfectly, then watching it catastrophically mis-forecast the 2022 bear market. The model “overfitted” to bull-market noise. Walk-forward testing with held-out periods is the industry-standard guard against this.

M3.030 Multiple Choice **HARD****Topic:** Forecast combination

You have three models forecasting BTC 7-day returns: Model A (RMSE = \$800), Model B (RMSE = \$750), Model C (RMSE = \$900). Which combined forecast is theoretically most likely to outperform all individual models?

- (A) Simple average: $(A + B + C)/3$
- (B) Weighted average inversely proportional to RMSE: $(w_A \cdot A + w_B \cdot B + w_C \cdot C)/(w_A + w_B + w_C)$ where $w_i = 1/\text{RMSE}_i$
- (C) Winner-takes-all: use Model B only (lowest RMSE)
- (D) Discard worst model: $(A + B)/2$ only

Solution

① **Correct Answer:** (B) — though (A) is often competitive in practice

② **Plain English**

Forecast combination works because different models make different errors — and when those errors are not perfectly correlated, averaging them causes partial cancellation. The theoretical optimum is inverse-variance weighting (which inverse-RMSE approximates). Simple averaging is remarkably effective in practice — the “forecast combination puzzle” in statistics shows that simple averages of diverse models beat the best individual model in the majority of real-world applications.

③ **Common Mistake**

Choosing (C) — winner-takes-all sounds logical but misses the diversification benefit. Model B might be best on average but make large errors in specific market conditions where Model A or C performs well. Combining captures this diversification. Empirically, combining N models almost always beats selecting the best one, even when you know which one was best in training.

④ **Real World Link**

Forecast combination is mainstream in financial forecasting — central banks combine econometric models with surveys and market prices. The Bank of England’s “fan chart” for inflation is literally a probability-weighted combination of multiple scenario forecasts. For BTC, combining ARIMA + Holt-Winters + a seasonal model almost always gives better MAPE than any single model alone.

M3.031 Write from Scratch **HARD****Topic:** Complete Holt-Winters model selection pipeline

Write a reusable function `hw_model_select(series, name)` that: (1) tests all four HW variants (SES, Holt trend, HW add seasonal, HW mul seasonal), (2) for each fits and records AIC, (3) selects winner by AIC, (4) plots all four 24-step forecasts on one chart with winner highlighted, (5) returns the best-fit object and results DataFrame.

Solution

① Correct Code

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import warnings
7 warnings.filterwarnings('ignore')
8 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
9
10 def hw_model_select(series, name="Time Series",
   seasonal_periods=12, forecast_h=24):
11     """Fit all four HW variants and select best by AIC."""
12     variants = [
13         {"label": "SES (no trend, no seasonal)",
14          "trend": None, "seasonal": None},
15         {"label": "Holt (add trend, no seasonal)",
16          "trend": "add", "seasonal": None},
17         {"label": "HW Additive (add trend + add seasonal)",
18          "trend": "add", "seasonal": "add"},
19         {"label": "HW Multiplicative (add trend + mul
   seasonal)",
20          "trend": "add", "seasonal": "mul"},
21     ]
22     results = []; fits = {}; forecasts = {}
23     colors = ["gray", "blue", "orange", "green"]
24
25     for v in variants:
26         try:
27             if v["seasonal"] is None:
28                 model = ExponentialSmoothing(series,
   trend=v["trend"])
29             else:
30                 model = ExponentialSmoothing(series,
   trend=v["trend"],
31                 seasonal=v["seasonal"],
   seasonal_periods=seasonal_periods)
32             fit = model.fit(optimized=True, disp=False)
33             fc = fit.forecast(forecast_h)
34             fits[v["label"]] = fit; forecasts[v["label"]]
   = fc
35             results.append({"Model": v["label"],
36                            "Alpha":
37                                round(fit.params.get("smoothing_level",
   float("nan")), 4),

```

```

        float("nan")), 4),
38         "Gamma":
            round(fit.params.get("smoothing_seasonal",
            float("nan")), 4),
39         "AIC": round(fit.aic, 2)})
40     except Exception:
41         results.append({"Model": v["label"], "AIC":
            float("inf")})
42
43     results_df =
44         pd.DataFrame(results).sort_values("AIC").reset_index(drop=True)
45     best_label = results_df.iloc[0]["Model"]
46     best_fit = fits.get(best_label)
47
48     # Plot
49     fig, ax = plt.subplots(figsize=(16, 6))
50     ax.plot(series[-3*seasonal_periods:],
51             color="steelblue",
52             linewidth=1.5, label="Historical", zorder=5)
53     for (label, fc), color in zip(forecasts.items(),
54                                 colors):
55         lw = 3 if label == best_label else 1.2
56         ls = "-" if label == best_label else "--"
57         sfx = " BEST" if label == best_label else ""
58         ax.plot(fc, color=color, linewidth=lw,
59                 linestyle=ls, label=label + sfx)
60     ax.axvline(x=series.index[-1], color="gray",
61                linestyle=":", linewidth=1.5)
62     ax.set_title(f"HW Variant Comparison -- {name}\nBest:
63                 {best_label}")
64     ax.set_ylabel("Value"); ax.legend(fontsize=8);
65     plt.tight_layout(); plt.show()
66
67     print(f"\nHolt-Winters Model Selection -- {name}")
68     print("="*70)
69     print(results_df[["Model", "Alpha", "Beta", "Gamma", "AIC"]].to_string(i
70     print("="*70); print(f"Best: {best_label}
71         AIC={results_df.iloc[0]['AIC']}")
72
73     return best_fit, results_df
74
75 # Apply to all three course assets
76 gold_m = yf.download("GC=F", start="2019-01-01",
77                       end="2024-12-31")["Close"].dropna().resample("ME").last()
78 btc_m = yf.download("BTC-USD", start="2019-01-01",
79                     end="2024-12-31")["Close"].dropna().resample("ME").last()
80 mrf_m = yf.download("MRF.NS", start="2019-01-01",
81                     end="2024-12-31")["Close"].dropna().resample("ME").last()

```

```

72 gold_best, _ = hw_model_select(gold_m, "Gold Futures")
73 btc_best, _ = hw_model_select(btc_m, "Bitcoin")
74 mrf_best, _ = hw_model_select(mrf_m, "MRF Tyres")

```

② Plain English

This function systematically evaluates all four Holt-Winters configurations and selects the best by AIC — automating what would otherwise be four separate model fits, four AIC checks, and four plots. Reusing this function on all three course assets reveals which seasonal structure each asset actually exhibits.

③ Common Mistake

Not handling the case where multiplicative seasonality fails — if the series has any zero or negative values, `seasonal="mul"` will crash. The `try/except` wrapper assigns `AIC=inf` for failed fits, ensuring they lose the competition gracefully without crashing the entire pipeline.

④ Real World Link

This function is the practical starting point for any new asset's time series analysis. In 30 seconds you get a defensible, AIC-ranked comparison of all standard exponential smoothing models. In professional settings, this is the “30-minute first look” before investing time in more complex ARIMA or SARIMA models.

M3.032 Multiple Choice **HARD**

Topic: MAPE failure modes

A quant analyst computes $\text{MAPE} = 340\%$ for their BTC daily return forecast. Which is the most likely explanation?

- (A) The model is extremely poor — 340% MAPE means forecasts are $3.4\times$ the actual values
- (B) MAPE is unreliable here because daily returns are close to zero — small actual values cause the percentage error to explode
- (C) The analyst made a coding error — MAPE cannot exceed 100% by definition
- (D) BTC is too volatile for any meaningful evaluation metric

Solution

① Correct Answer: (B)

```

1 import numpy as np
2
3 # Demonstration of MAPE failure on near-zero series
4 actual = np.array([ 0.001, -0.002,  0.0005, -0.001,
5                    0.003])
6 forecast = np.array([ 0.002,  0.001, -0.0005,  0.002,
7                    -0.001])
8 errors = actual - forecast
9 mape = np.mean(np.abs(errors / actual)) * 100

```

```

8 print(f"MAPE: {mape:.1f}%") # will be >> 100% due to
   near-zero actual values
9 # Issue: when actual=0.0005 and error=0.001, pct error =
   200%
10
11 # REMEDY: Use MAE or RMSE for near-zero return series
12 mae = np.mean(np.abs(errors))
13 rmse = np.sqrt(np.mean(errors**2))
14 print(f"MAE = {mae:.6f} (interpretable for return
   series)")
15 print(f"RMSE = {rmse:.6f} (interpretable for return
   series)")

```

② Plain English

MAPE divides by the actual value — when actual returns are tiny (say 0.001 = 0.1%), even a small absolute error (say 0.002) gives $\text{MAPE} = 200\%$. This is mathematically correct but practically useless — the forecast was only 0.2 percentage points off, which is excellent, but MAPE reports it as a disaster. MAPE is designed for price levels (large positive numbers), not return series near zero.

③ Common Mistake

Choosing (C) — MAPE absolutely can exceed 100%. MAPE is $\text{mean}(|\text{error}/\text{actual}|) \times 100$, and if the absolute error exceeds the actual value, the percentage exceeds 100%. This is perfectly valid mathematically. MAPE's upper bound is only 100% if forecasts are constrained to be non-negative and the series is non-negative — which returns are not.

④ Real World Link

Professional forecasters always specify which metric they use and why. For BTC price level forecasts: use MAPE (values are large, positive, meaningful as percentages). For BTC daily return forecasts: use MAE or RMSE with explicit units ("MAE = 0.002 log return units $\approx$ 0.2% per day"). Choosing the wrong metric undermines the credibility of any analysis.

M3.033 Predict the Output **HARD**

Topic: AIC-BIC disagreement — sample size effect

Without running this code, predict which model each criterion selects, and explain why they disagree.

```

1 import numpy as np
2 from statsmodels.tsa.arima.model import ARIMA
3
4 np.random.seed(10)
5 n, phi = 1500, 0.3
6 y = np.zeros(n)
7 for t in range(1, n):
8     y[t] = phi * y[t-1] + np.random.randn()
9

```

```

10 fit1 = ARIMA(y, order=(1, 0, 0)).fit()
11 fit3 = ARIMA(y, order=(3, 0, 0)).fit()
12
13 print(f"AR(1): AIC={fit1.aic:.2f}, BIC={fit1.bic:.2f}")
14 print(f"AR(3): AIC={fit3.aic:.2f}, BIC={fit3.bic:.2f}")
15 print(f"AIC selects: {'AR(1)' if fit1.aic < fit3.aic else
      'AR(3)'}")
16 print(f"BIC selects: {'AR(1)' if fit1.bic < fit3.bic else
      'AR(3)'}")

```

Predict: which model does AIC select? BIC? Do they agree?

Solution

① Predicted Output

```

AR(1): AIC=[lower or close], BIC=[lower]
AR(3): AIC=[slightly higher], BIC=[higher by more]
AIC selects: AR(1) [probably, true order is 1]
BIC selects: AR(1) [more definitively]

```

```

1 # Reasoning:
2 # True model is AR(1) with phi=0.3 -- AR lags 2 and 3 are
  # genuinely zero
3 # Adding AR(2) and AR(3) terms captures only noise
4 #
5 # AIC: Adding 2 parameters -> penalty = +2x2 = +4
6 # Improvement in -2xlogL from fitting noise terms on
  # n=1500: very small
7 # Net AIC ~ = +4 - small = positive -> AIC prefers AR(1)
8 #
9 # BIC: Adding 2 parameters -> penalty = 2xlog(1500) =
  # 2x7.31 = +14.6
10 # Even if noise terms improve logL slightly, +14.6 >>
  # small improvement
11 # BIC >> 0 -> BIC STRONGLY prefers AR(1)
12 #
13 # Both agree (true order = 1), but BIC agrees much more
  # strongly
14 # AIC gap might be 2-4 units; BIC gap might be 12-16 units
15 print(f"log(1500) = {np.log(1500):.2f}")
16 print(f"BIC/AIC penalty ratio per extra parameter:
      {np.log(1500)/2:.2f}x")

```

② Plain English

When the true model is AR(1), both AIC and BIC should select AR(1) — the extra parameters in AR(3) capture only noise. The key difference: BIC's agreement is far

more decisive (larger gap). For $n = 1500$, BIC penalises each spurious AR term by 7.3 units vs AIC's 2 units — BIC “knows” it has enough data to be confident that unnecessary parameters are unnecessary.

③ Common Mistake

Expecting AIC and BIC to always disagree. They agree when the evidence for or against extra parameters is clear-cut. They disagree most strongly when models have similar log-likelihood and BIC's stronger penalty tips the balance toward the simpler model. In this AR(1) case with $n = 1500$, both clearly prefer AR(1) — but BIC does so with a much larger margin.

④ Real World Link

For BTC daily returns with $n \approx 1250$ (similar to $n = 1500$ here), BIC's large-sample conservatism means it will almost always pick the simplest adequate model — typically AR(1) or even ARMA(0,0). This is often the right call for efficient market returns. AIC sometimes goes to AR(2) or ARMA(1,1), which may or may not improve walk-forward accuracy.

M3.034 Short Answer **HARD**

Topic: Forecast variance grows with horizon

In 2 sentences, explain mathematically why forecast uncertainty (variance of forecast error) grows with the forecast horizon h for an AR(1) model, and state the implication for long-horizon BTC forecasts.

Solution

① Model Answer

For an AR(1) model $X_t = \varphi \cdot X_{t-1} + \varepsilon_t$, the h -step-ahead forecast error variance is $\sigma^2(1 + \varphi^2 + \varphi^4 + \dots + \varphi^{2(h-1)}) = \sigma^2 \cdot (1 - \varphi^{2h})/(1 - \varphi^2)$, which grows from σ^2 at $h = 1$ and converges to the process variance $\sigma^2/(1 - \varphi^2)$ as $h \rightarrow \infty$ — meaning uncertainty increases with horizon until it hits the unconditional variance ceiling. For BTC forecasts, this means 1-day confidence intervals are narrow (based mostly on today's well-known starting position), while 30-day intervals are nearly as wide as the unconditional distribution of BTC returns — effectively saying the model provides minimal information beyond a few days horizon.

② Common Mistake

Thinking forecast variance grows without bound. It doesn't — for stationary AR(1) with $|\varphi| < 1$, variance converges to the process variance as $h \rightarrow \infty$. Only for unit root ($\varphi = 1$, random walk) does variance grow linearly without bound: $\text{Var}(h) = h \cdot \sigma^2$. This distinction matters: AR(1) forecasts become useless but bounded, random walk forecasts become increasingly uncertain without limit.

③ Real World Link

This variance ceiling explains why crypto options pricing doesn't scale linearly with time — a 30-day BTC option isn't $30\times$ more uncertain than a 1-day option, it's more like $\sqrt{30} \approx 5.5\times$ (square root of time rule). Understanding AR forecast variance growth is directly applicable to BTC derivatives pricing and risk management.

Section D ★ Exam Style Problems — M3.035 to M3.040

M3.035 Exam Style — Holt-Winters Full Pipeline on Gold Monthly EXAM

Marks: 20 | Time: 35 minutes | Dataset: GC=F Monthly

Part A — Data Preparation and Stationarity Context [4 marks]

Download Gold futures (GC=F, 2019–2024), resample to monthly. Plot the series. In a comment, state whether Holt-Winters requires stationarity and why this matters for model selection.

Part B — Model Fitting and Parameter Interpretation [5 marks]

Fit Holt-Winters with additive trend and multiplicative seasonality. Print all three smoothing parameters (α, β, γ) . Write a 3-line comment interpreting what each parameter value means for Gold's price behaviour.

Part C — 18-Month Forecast with Confidence Intervals [5 marks]

Generate an 18-month forecast. Produce confidence intervals using Monte Carlo simulation (500 paths). Plot historical + forecast + 80% and 95% CI bands in two shades.

Part D — Residual Analysis [6 marks]

Compute model residuals. Plot ACF of residuals and a histogram. Run Ljung-Box test (15 lags). Write a structured 4-point conclusion on model adequacy.

Solution

① Complete Solution

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, scipy.stats as spstats
4 import warnings; warnings.filterwarnings('ignore')
5 from statsmodels.tsa.holtwinters import
    ExponentialSmoothing
6 from statsmodels.graphics.tsaplots import plot_acf
7 from statsmodels.stats.diagnostic import acorr_ljungbox
8
9 # PART A -- Data Preparation [4 marks]
```

```

10 gold      = yf.download("GC=F", start="2019-01-01",
11      end="2024-12-31")
12 monthly = gold["Close"].dropna().resample("ME").last()
13 print(f"PART A | Observations: {len(monthly)} | Range: "
14      f"{monthly.index[0].date()} to
15      {monthly.index[-1].date()}")
16 print(f"Price: ${monthly.min():,.2f} to
17      ${monthly.max():,.2f}")
18
19 plt.figure(figsize=(14, 4))
20 monthly.plot(color="gold", linewidth=2)
21 plt.title("PART A -- Gold Futures Monthly Closing Price");
22 plt.tight_layout(); plt.show()
23
24 # Holt-Winters does NOT require stationarity -- it models
25 # the RAW LEVEL series.
26 # Unlike ARIMA (which requires stationarity via
27 # differencing), HW handles trend and
28 # seasonality explicitly through level/trend/seasonal
29 # smoothing components.
30 # This makes HW appropriate for Gold's non-stationary,
31 # trending, seasonal price series.
32
33 #          PART B -- Model Fitting and Parameters [5 marks]
34
35 model = ExponentialSmoothing(monthly, trend="add",
36      seasonal="mul",
37      seasonal_periods=12)
38 fit = model.fit(optimized=True)
39 print(fit.summary())
40
41 alpha = fit.params["smoothing_level"]
42 beta  = fit.params["smoothing_trend"]
43 gamma = fit.params["smoothing_seasonal"]
44 print(f"\nAlpha (alpha={alpha:.4f}): "
45      f"{'LOW -- level smoothed heavily; recent months
46      dont dominate' if alpha < 0.5 else 'HIGH -- level
47      adapts quickly'}")
48 print(f"Beta (beta={beta:.4f}) : "
49      f"{'LOW -- trend is stable and changes very slowly
50      over time' if beta < 0.1 else 'MODERATE -- trend
51      updates with responsiveness'}")
52 print(f"Gamma (gamma={gamma:.4f}): "
53      f"Moderate -- seasonal factors update with each new
54      annual cycle")
55 # 1. Low alpha: Gold's monthly level is heavily smoothed;
56 # spikes don't dominate

```

```

42 # 2. Very low beta: Gold's upward trend from $1300 to
    $2700 is gradual and stable
43 # 3. Moderate gamma: Seasonal factors (Diwali, Chinese New
    Year) are stable year-to-year
44
45 #          PART C -- 18-Month Forecast with CI [5 marks]
46 h = 18; fc_point = fit.forecast(h)
47 simulations = fit.simulate(nsimulations=h,
    repetitions=500, error="mul")
48
49 ci_80_lo = simulations.quantile(0.10, axis=1)
50 ci_80_hi = simulations.quantile(0.90, axis=1)
51 ci_95_lo = simulations.quantile(0.025, axis=1)
52 ci_95_hi = simulations.quantile(0.975, axis=1)
53
54 fig, ax = plt.subplots(figsize=(16, 6))
55 monthly.plot(ax=ax, color="steelblue", linewidth=1.5,
    label="Historical Gold")
56 ax.plot(fc_point, color="red", linewidth=2.5,
    label="18-Month Forecast")
57 ax.fill_between(fc_point.index, ci_95_lo, ci_95_hi,
    color="red", alpha=0.15, label="95% CI")
58 ax.fill_between(fc_point.index, ci_80_lo, ci_80_hi,
    color="red", alpha=0.30, label="80% CI")
59 ax.axvline(monthly.index[-1], color="gray",
    linestyle="--", linewidth=1.5)
60
61 ax.set_title("PART C -- Gold Futures: HW 18-Month
    Forecast")
62 ax.set_ylabel("USD/oz"); ax.legend(fontsize=9);
    plt.tight_layout(); plt.show()
63
64
65 #          PART D -- Residual Analysis [6 marks]
66 resid = (monthly - fit.fittedvalues).dropna()
67 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
68 fig.suptitle("PART D -- HW Residual Analysis: Gold
    Monthly", fontsize=12)
69
70 axes[0].plot(resid, color="steelblue", linewidth=1)
71 axes[0].axhline(0, color="red", linestyle="--")
72 axes[0].set_title("Residuals Over Time")
73
74 plot_acf(resid, lags=15, alpha=0.05, ax=axes[1],
    zero=False)
75 axes[1].set_title("ACF of Residuals (want: all inside
    bands)")
76

```

```

77 axes[2].hist(resid, bins=20, density=True,
    color="steelblue", alpha=0.6)
78 x = np.linspace(resid.min(), resid.max(), 200)
79 axes[2].plot(x, spstats.norm.pdf(x, resid.mean(),
    resid.std()), "r-", lw=2)
80 axes[2].set_title("Residual Distribution")
81 plt.tight_layout(); plt.show()
82
83 lb = acorr_ljungbox(resid, lags=15, return_df=True)
84 n_sig = (lb["lb_pvalue"] < 0.05).sum()
85 print(f"\nPART D -- Ljung-Box (15 lags): {n_sig}
    significant lags")
86 print("4-Point Conclusion:")
87 print(f"  1. MEAN: {n_sig}/15 significant ACF lags -- "
88       f"'well-captured' if n_sig==0 else 'some structure
    remains'")
89 print(f"  2. DISTRIBUTION: Residuals approximately normal;
    mild tails typical for commodities.")
90 print(f"  3. VARIANCE: COVID period (2020) may show larger
    residuals -- ARCH effects possible.")
91 print(f"  4. RECOMMENDATION: HW captures Gold trend and
    seasonality adequately. "
92       f"Complement with GARCH on residuals for full risk
    modelling (Module 4).")

```

④ Real World Link

This complete HW analysis of Gold monthly prices is exactly what a commodity research analyst would produce before presenting an 18-month Gold price outlook to portfolio managers. The confidence intervals translate directly to scenario planning: the 80% CI gives the “base case range,” the 95% CI gives the “stress scenario range” used in risk management.

M3.036 Exam Style — Walk-Forward Validation and Model Comparison **EXAM**

Marks: 20 | Time: 35 minutes | Dataset: BTC-USD

Part A — Benchmark Establishment

[4 marks]

Implement naïve random walk benchmark for BTC (last-value forecast). Compute RMSE and MAPE over the last 180 trading days.

Part B — Holt’s Model Evaluation

[5 marks]

Implement 1-step-ahead walk-forward validation for Holt’s double exponential smoothing (expanding window, min 252 days). Compute RMSE and MAPE. Compare to naïve benchmark.

Part C — Information Ratio

[5 marks]

Compute the “Information Ratio” of forecast errors: $IR = \bar{d}/\text{std}(d)$ where $d_t = e_{\text{naive},t}^2 - e_{\text{holt},t}^2$ is the loss differential. Positive IR means Holt’s systematically outperforms. Plot the cumulative loss differential.

Part D — Conclusion and Reporting*[6 marks]*

Write a structured model evaluation report as a formatted print block covering: dataset, benchmark, model, evaluation period, RMSE, MAPE, IR, and whether Holt's is deployment-ready.

Solution**① Complete Solution**

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, matplotlib.pyplot as
   plt
3 import warnings; warnings.filterwarnings('ignore')
4 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
5
6 btc    = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
7 close = btc["Close"].dropna().values
8 n      = len(close)
9 test_window = 180; split = n - test_window
10
11 #          PART A -- Naive Benchmark [4 marks]
12
13 train_bench = close[:split]; test_bench = close[split:]
14 naive_fc    = np.full(test_window, train_bench[-1])
15 e_naive     = test_bench - naive_fc
16 rmse_naive  = np.sqrt(np.mean(e_naive**2))
17 mape_naive  = np.mean(np.abs(e_naive / test_bench)) * 100
18 print(f"PART A -- Naive: RMSE=${rmse_naive:,.2f} |
   MAPE={mape_naive:,.2f}%")
19
20 #          PART B -- Holt's Walk-Forward [5 marks]
21
22 min_train = 252; holt_fc = []; holt_act = []
23 for t in range(min_train, n - 1):
24     train = close[0:t]
25     try:
26         fit = ExponentialSmoothing(train, trend="add",
27                                     damped_trend=True).fit(optimized=True,
28                                                             disp=False)
29         holt_fc.append(fit.forecast(1)[0]);
30         holt_act.append(close[t])
31     except Exception: continue
32
33 holt_fc = np.array(holt_fc); holt_act = np.array(holt_act)
34 e_holt  = holt_act - holt_fc
35 e_holt_180 = e_holt[-test_window:]; act_180 =
   holt_act[-test_window:]

```

```

32
33 rmse_holt = np.sqrt(np.mean(e_holt_180**2))
34 mape_holt = np.mean(np.abs(e_holt_180 / act_180)) * 100
35 print(f"PART B -- Holt's: RMSE=${rmse_holt:,.2f} |
      MAPE={mape_holt:.2f}%")
36 print(f"Holt beats naive: {rmse_holt < rmse_naive}")
37
38 #          PART C -- Information Ratio [5 marks]
39 e_naive_180 = test_bench -
      np.concatenate([[train_bench[-1]], test_bench[:-1]])
40 d_t = e_naive_180**2 - e_holt_180**2
41 d_bar = np.mean(d_t); d_std = np.std(d_t, ddof=1)
42 IR     = d_bar / d_std if d_std > 0 else 0.0
43
44 fig, axes = plt.subplots(2, 1, figsize=(14, 8))
45 cumulative_d = np.cumsum(d_t)
46 axes[0].plot(cumulative_d, color="purple", linewidth=1.5)
47 axes[0].axhline(0, color="black", linestyle="--")
48 axes[0].fill_between(range(len(cumulative_d)),
      cumulative_d, 0,
49                      where=(cumulative_d > 0),
                        color="green", alpha=0.2,
                        label="Holt leads")
50 axes[0].fill_between(range(len(cumulative_d)),
      cumulative_d, 0,
51                      where=(cumulative_d < 0),
                        color="red", alpha=0.2,
                        label="Naive leads")
52 axes[0].set_title("Cumulative Loss Differential (Holt vs
      Naive)")
53 axes[0].legend()
54
55 axes[1].plot(test_bench[-60:], color="steelblue",
      label="Actual BTC")
56 axes[1].plot(holt_fc[-60:], color="red",
      label="Holt Forecast")
57 axes[1].set_title("Last 60 Days: Actual vs Holt
      Forecast"); axes[1].legend()
58 plt.tight_layout(); plt.show()
59
60 print(f"\nPART C -- IR = {IR:.4f} | "
      f"'Holt outperforms' if d_bar > 0 else 'Naive
      outperforms' on average")
61
62 #          PART D -- Evaluation Report [6 marks]
63
64 deploy = (rmse_holt < rmse_naive) and (mape_holt <
      mape_naive)

```

```

67 print(f"""
68 {'='*62}
69     MODEL EVALUATION REPORT | Time Series Analytics T6725
70 {'='*62}
71     DATASET      : Bitcoin (BTC-USD), Daily, 2019-01-01 to
72                   2024-12-31
73     EVAL PERIOD: Last {test_window} trading days (held-out)
74     TRAIN MIN   : {min_train} days (expanding window)
75
76     BENCHMARK   : Naive Random Walk (last value forward)
77                   RMSE=${rmse_naive:,.2f} /
78                   MAPE={mape_naive:.2f}%
79
80     CANDIDATE    : Holt's Double Exponential Smoothing (damped)
81                   Walk-forward steps: {len(e_holt)}
82                   RMSE=${rmse_holt:,.2f} /
83                   MAPE={mape_holt:.2f}%
84
85     COMPARISON: RMSE reduction = ${rmse_naive-rmse_holt:,.2f}
86                 IR = {IR:.4f} ({'systematic outperformance'
87                               if d_bar>0 else 'inconsistent'})
88
89     VERDICT      : {'DEPLOYMENT READY: Holt outperforms naive
90                   on both metrics.' if deploy else 'NOT READY: Holt
91                   does not consistently beat naive benchmark.'}
92
93     NOTE         : GARCH model should complement this mean
94                   forecast (Module 4).
95 {'='*62}
96 """)

```

④ Real World Link

This evaluation report format is directly applicable to the “Model Validation” section required by financial regulators (Basel III Model Risk Management guidelines). Before any model touches real capital allocation at a bank or fund, a report with exactly these components must be signed off by independent model validators.

M3.037 Exam Style — AIC/BIC Grid Search and Forecast Comparison **EXAM**

Marks: 15 | Time: 30 minutes | Dataset: MRF.NS Monthly

Part A — ARIMA Grid Search [5 marks]

Fit ARIMA(p,1,q) for $p, q \in \{0, 1, 2\}$ on MRF monthly prices. Display AIC heatmap and BIC heatmap side-by-side. Highlight best cell in each.

Part B — Winner Diagnostics [5 marks]

Fit the AIC-winning model. Full residual diagnostics: ACF, Q-Q, Ljung-Box. State verdict.

Part C — 12-Month Forecast vs Holt-Winters [5 marks]

Fit Holt-Winters (add trend, add seasonal, period=12) on the same training data. Generate 12-month forecast from both models. Plot side-by-side. Compute and compare MAPE on a 12-month hold-out period.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, scipy.stats as spstats
4 import warnings; warnings.filterwarnings('ignore')
5 from statsmodels.tsa.arima.model import ARIMA
6 from statsmodels.tsa.holtwinters import
   ExponentialSmoothing
7 from statsmodels.graphics.tsaplots import plot_acf
8 from statsmodels.stats.diagnostic import acorr_ljungbox
9
10 mrf      = yf.download("MRF.NS", start="2019-01-01",
   end="2024-12-31")
11 monthly = mrf["Close"].dropna().resample("ME").last()
12 train_m = monthly[:-12]; test_m = monthly[-12:]
13
14 #          PART A -- Grid Search [5 marks]
15
16 orders = [0, 1, 2]
17 aic_grid = np.full((3,3), np.nan); bic_grid =
   np.full((3,3), np.nan)
18 fit_dict = {}
19
20 for i, p in enumerate(orders):
21     for j, q in enumerate(orders):
22         try:
23             fit = ARIMA(train_m, order=(p,1,q)).fit()
24             aic_grid[i,j] = fit.aic; bic_grid[i,j] =
   fit.bic
25             fit_dict[(p,q)] = fit
26         except: pass
27
28 best_aic_idx = np.unravel_index(np.nanargmin(aic_grid),
   aic_grid.shape)
29 best_bic_idx = np.unravel_index(np.nanargmin(bic_grid),
   bic_grid.shape)
30 best_aic_p, best_aic_q = orders[best_aic_idx[0]],
   orders[best_aic_idx[1]]
31 print(f"PART A -- Best AIC:
   ARIMA({best_aic_p},1,{best_aic_q}) =
   {aic_grid[best_aic_idx]:.2f}")

```

```

32 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
33 for ax, grid, title, best_idx in zip(axes, [aic_grid,
34                                     bic_grid],
35                                     ["AIC Grid", "BIC
36                                     Grid"],
37                                     [best_aic_idx,
38                                     best_bic_idx]):
39     im = ax.imshow(grid, cmap="RdYlGn_r", aspect="auto")
40     for i in range(3):
41         for j in range(3):
42             if not np.isnan(grid[i,j]):
43                 c = "white" if (i,j)==best_idx else "black"
44                 w = "bold" if (i,j)==best_idx else
45                     "normal"
46                 ax.text(j, i, f"{grid[i,j]:.1f}",
47                         ha="center", va="center",
48                         color=c, fontsize=11, fontweight=w)
49     ax.set_xticks(range(3)); ax.set_xticklabels([f"q={q}"
50         for q in orders])
51     ax.set_yticks(range(3)); ax.set_yticklabels([f"p={p}"
52         for p in orders])
53     ax.set_title(f"PART A -- {title}: ARIMA(p,1,q) MRF
54         Monthly")
55     plt.colorbar(im, ax=ax)
56 plt.tight_layout(); plt.show()
57
58 # PART B -- Diagnostics [5 marks]
59
60 best_fit = fit_dict[(best_aic_p, best_aic_q)]
61 resid = best_fit.resid.dropna()
62
63 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
64 fig.suptitle(f"PART B -- Diagnostics:
65     ARIMA({best_aic_p},1,{best_aic_q})", fontsize=12)
66 plot_acf(resid, lags=15, alpha=0.05, ax=axes[0],
67     zero=False)
68 axes[0].set_title("ACF of Residuals")
69 axes[1].hist(resid, bins=20, density=True,
70     color="steelblue", alpha=0.6)
71 x = np.linspace(resid.min(), resid.max(), 200)
72 axes[1].plot(x, spstats.norm.pdf(x, resid.mean(),
73     resid.std()), "r-", lw=2)
74 axes[1].set_title("Residual Distribution")
75 spstats.probplot(resid, dist="norm", plot=axes[2]);
76 axes[2].set_title("Q-Q Plot")
77 plt.tight_layout(); plt.show()
78
79 lb = acorr_ljungbox(resid, lags=15, return_df=True)
80 n_sig = (lb["lb_pvalue"] < 0.05).sum()

```

```

67 print(f"PART B -- Ljung-Box: {n_sig}/15 significant | "
68       f"{'White noise residuals' if n_sig==0 else
        'Residual autocorrelation'}")
69
70 #           PART C -- Forecast vs HW [5 marks]
71
72 arima_fc = best_fit.forecast(12)
73 hw_fit    = ExponentialSmoothing(train_m, trend="add",
74                                   seasonal="add",
75                                   seasonal_periods=12).fit(optimized=True)
76 hw_fc     = hw_fit.forecast(12)
77
78 mape_arima = np.mean(np.abs((test_m.values -
79                               arima_fc.values) / test_m.values)) * 100
80 mape_hw    = np.mean(np.abs((test_m.values - hw_fc.values)
81                               / test_m.values)) * 100
82
83 print(f"\nPART C -- MAPE:
84       ARIMA({best_aic_p},1,{best_aic_q})={mape_arima:.2f}%"
85       f" | HW={mape_hw:.2f}%")
86
87 print(f"Winner: {'ARIMA' if mape_arima < mape_hw else
88       'Holt-Winters'}")
89
90 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
91 for ax, fc, name in zip(axes, [arima_fc, hw_fc],
92                           [f"ARIMA({best_aic_p},1,{best_aic_q})",
93                            "Holt-Winters"]):
94     ax.plot(monthly[-24:], color="steelblue",
95             linewidth=1.5, label="Historical")
96     ax.plot(test_m, color="black", linewidth=1.5,
97             linestyle="--", label="Actual Test")
98     ax.plot(fc, color="red", linewidth=2, label=f"{name}
99             Forecast")
100     mape_v = mape_arima if "ARIMA" in name else mape_hw
101     ax.set_title(f"{name}\nTest MAPE: {mape_v:.2f}%");
102     ax.legend(fontsize=9)
103 fig.suptitle("PART C -- ARIMA vs HW: 12-Month MRF
104             Forecast", fontsize=12)
105 plt.tight_layout(); plt.show()

```

M3.038 Exam Style — Full Forecast Evaluation Suite EXAM
Marks: 25 | Time: 50 minutes | Dataset: BTC-USD and GC=F

Part A (5 marks): Download BTC daily and Gold monthly. Compute log returns for BTC, level for Gold. Confirm stationarity for BTC returns only (ADF). Document why Gold level is acceptable for HW without ADF.

Part B (5 marks): Fit four models: BTC SES, BTC Holt damped, Gold HW add+mul, Gold HW mul+mul. Record AIC for Gold models. Compute RMSE on

90-day (BTC) and 12-month (Gold) hold-out test sets.

Part C (5 marks): Walk-forward RMSE for BTC Holt's with `min_train=252`. Plot walk-forward errors distribution and cumulative squared error vs naïve benchmark.

Part D (5 marks): Combine BTC SES + Holt forecasts (simple average). Show whether combination beats both individual models.

Part E (5 marks): Write a structured 12-line analysis memo as a comment block, comparing all models across both assets, citing metrics, and making specific deployment recommendations.

Solution

① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, scipy.stats as spstats
4 import warnings; warnings.filterwarnings('ignore')
5 from statsmodels.tsa.holtwinters import
   SimpleExpSmoothing, ExponentialSmoothing
6 from statsmodels.tsa.stattools import adfuller
7
8 #          PART A -- Data Preparation [5 marks]
9
10 btc  = yf.download("BTC-USD", start="2019-01-01",
11                   end="2024-12-31")
12 gold = yf.download("GC=F",      start="2019-01-01",
13                   end="2024-12-31")
14
15 btc_close      = btc["Close"].dropna()
16 btc_returns    = np.log(btc_close).diff().dropna()
17 gold_monthly   = gold["Close"].dropna().resample("ME").last()
18
19 adf_btc = adfuller(btc_returns)
20 print(f"PART A | BTC returns n={len(btc_returns)} ADF
21       p={adf_btc[1]:.6f} "
22       f"-> {'Stationary' if adf_btc[1]<0.05 else
23           'Non-stationary'}")
24
25 print(f"          | Gold monthly n={len(gold_monthly)} obs")
26 # HW does NOT require stationarity: it explicitly models
27   level, trend, and seasonal
28 # components via recursive updates -- designed inherently
29   for non-stationary trending series.
30
31 btc_train = btc_returns.iloc[:-90]; btc_test =
32   btc_returns.iloc[-90:]
33 gold_train = gold_monthly.iloc[:-12]; gold_test =
34   gold_monthly.iloc[-12:]
35

```

```

26 # PART B -- Four Model Fits + RMSE [5 marks]

27 ses_btc =
    SimpleExpSmoothing(btc_train).fit(optimized=True)
28 hw_btc = ExponentialSmoothing(btc_train, trend="add",
29                               damped_trend=True).fit(optimized=True)
30 gold_aa = ExponentialSmoothing(gold_train, trend="add",
31                                seasonal="add",
32                                seasonal_periods=12).fit(optimized=True)
33 gold_am = ExponentialSmoothing(gold_train, trend="add",
34                                seasonal="mul",
35                                seasonal_periods=12).fit(optimized=True)

36 fc_ses = ses_btc.forecast(90); fc_hw = hw_btc.forecast(90)
37 fc_aa = gold_aa.forecast(12); fc_am = gold_am.forecast(12)

38 def rmse_fn(a, f): return
    np.sqrt(np.mean((np.array(a)-np.array(f))**2))

39
40 r_ses = rmse_fn(btc_test, fc_ses); r_hw =
    rmse_fn(btc_test, fc_hw)
41 r_aa = rmse_fn(gold_test, fc_aa); r_am =
    rmse_fn(gold_test, fc_am)

42
43 print(f"\nPART B | BTC SES RMSE={r_ses:.6f} | BTC Holt
    RMSE={r_hw:.6f}")
44 print(f"          | Gold add+add AIC={gold_aa.aic:.1f}
    RMSE=${r_aa:,.2f}")
45 print(f"          | Gold add+mul AIC={gold_am.aic:.1f}
    RMSE=${r_am:,.2f}")
46 print(f"          | Gold winner: {'add+mul' if r_am < r_aa
    else 'add+add'} by test RMSE")

47
48 # PART C -- Walk-Forward [5 marks]

49 btc_arr = btc_returns.values
50 btc_holt_wf = []; btc_act_wf = []
51 for t in range(252, len(btc_arr)-1):
52     try:
53         fit = ExponentialSmoothing(btc_arr[0:t],
54                                     trend="add",
55                                     damped_trend=True).fit(optimized=True,
56                                                             disp=False)
57         btc_holt_wf.append(fit.forecast(1)[0]);
58         btc_act_wf.append(btc_arr[t])
59     except Exception: continue

60 holt_wf = np.array(btc_holt_wf); act_wf =
    np.array(btc_act_wf)

```

```

59 naive_wf = np.concatenate([[btc_arr[251]],
    act_wf[:-1]])[:len(holt_wf)]
60 e_holt_wf = act_wf - holt_wf; e_naive_wf = act_wf -
    naive_wf
61
62 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
63 axes[0].hist(e_holt_wf, bins=60, density=True,
    color="steelblue", alpha=0.7)
64 x = np.linspace(e_holt_wf.min(), e_holt_wf.max(), 300)
65 axes[0].plot(x, spstats.norm.pdf(x, e_holt_wf.mean(),
    e_holt_wf.std()), "r-", lw=2)
66 axes[0].set_title("PART C -- Walk-Forward Error
    Distribution")
67 axes[1].plot(np.cumsum(e_holt_wf**2), label="Holt's",
    color="red")
68 axes[1].plot(np.cumsum(e_naive_wf**2), label="Naive",
    color="gray", linestyle="--")
69 axes[1].set_title("Cumulative Squared Error: Holt vs
    Naive"); axes[1].legend()
70 plt.tight_layout(); plt.show()
71
72 print(f"\nPART C | Steps={len(e_holt_wf)} | "
73       f"Holt RMSE={np.sqrt(np.mean(e_holt_wf**2)):.6f} | "
74       f"Naive RMSE={np.sqrt(np.mean(e_naive_wf**2)):.6f}")
75
76 #          PART D -- Forecast Combination [5 marks]
77
78 fc_combo = (fc_ses.values + fc_hw.values) / 2
79 r_combo = rmse_fn(btc_test, fc_combo)
80 print(f"\nPART D | SES={r_ses:.6f} | Holt={r_hw:.6f} |
    Combined={r_combo:.6f}")
81 print(f"Combination beats SES : {r_combo < r_ses}")
82 print(f"Combination beats HW : {r_combo < r_hw}")
83 print(f"Both beaten : {r_combo < r_ses and
    r_combo < r_hw}")
84
85 #          PART E -- Analysis Memo [5 marks]
86
87 gold_w = 'add+mul' if r_am < r_aa else 'add+add'
88 btc_w = 'Combined' if r_combo < min(r_ses, r_hw) else
    ('SES' if r_ses < r_hw else 'Holt')
89 print(f"")
90 #
91
92 # ANALYSIS MEMO -- Time Series Analytics T6725 Module 3
93 #
94
95 # SCOPE: Forecast model evaluation for BTC-USD (daily
    returns)

```

```

92 #           and Gold Futures GC=F (monthly prices), 2019-2024.
93 #
94 # BTC: Daily log returns (n~1250) are stationary (ADF
    p<0.001).
95 #   SES RMSE={r_ses:.6f} | Holt RMSE={r_hw:.6f} |
    Combined={r_combo:.6f}.
96 #   {btc_w} is marginal winner; gap small -- returns near
    white noise.
97 #   Walk-forward confirms Holt marginally outperforms
    naive.
98 #   Recommendation: Deploy combined SES+Holt; run DM test
    for significance.
99 #   Note: GARCH required for volatility -- mean model
    alone insufficient.
100 #
101 # GOLD: Monthly prices (n~72) non-stationary; HW
    appropriate.
102 #   Gold {gold_w} wins by AIC and test RMSE.
103 #   Recommendation: Deploy HW {gold_w} for Gold 12-month
    forecasting.
104 #   Confidence intervals via Monte Carlo simulation for
    risk planning.
105 #
106 # OVERALL: Simple exponential smoothing models provide
    reasonable baselines.
107 #   GARCH (Module 4) needed for complete risk modelling of
    both assets.
108 #
109 """)

```

M3.039 Exam Style — Durbin-Levinson, FPE, and Model Estimation EXAM
Marks: 15 | Time: 25 minutes | Dataset: BTC-USD

Part A (5 marks): Manually compute sample mean, variance, and ACF at lags 1, 2, 3 for BTC log returns. Compare with `acf()` output. Show working.

Part B (5 marks): Implement FPE criterion for AR order $p = 0$ to $p = 6$ on BTC returns. Plot FPE vs p . Identify optimal p . Compare with AIC-selected order.

Part C (5 marks): Using the FPE-optimal AR order, fit the model, check residual ACF, and produce a 10-day forecast with a comment explaining why the forecast quickly flattens near zero.

Solution
① Complete Solution

```

1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, matplotlib.pyplot as
   plt
3 import warnings; warnings.filterwarnings('ignore')
4 from statsmodels.tsa.stattools import acf
5 from statsmodels.tsa.ar_model import AutoReg
6 from statsmodels.tsa.arima.model import ARIMA
7 from statsmodels.graphics.tsaplots import plot_acf
8
9 btc      = yf.download("BTC-USD", start="2019-01-01",
   end="2024-12-31")
10 returns = np.log(btc["Close"]).diff().dropna()
11 r        = returns.values; n = len(r)
12
13 #          PART A -- Manual ACF [5 marks]
14
15 mu_hat   = np.mean(r); var_hat = np.var(r, ddof=0)
16 print(f"PART A | n={n} | mean={mu_hat:.8f} |
   var={var_hat:.8f} | std={np.sqrt(var_hat):.6f}")
17
18 def manual_autocov(s, k):
19     mu = np.mean(s)
20     return np.sum([(s[t]-mu)*(s[t-k]-mu) for t in range(k,
   len(s))]) / len(s)
21
22 def manual_autocorr(s, k):
23     return manual_autocov(s, k) / manual_autocov(s, 0)
24
25 acf_sm = acf(r, nlags=3, fft=True)
26 print(f"\n  {'Lag':>4}  {'Manual':>12}
   {'Statsmodels':>12}  {'Match':>6}")
27 for k in range(4):
28     m = manual_autocorr(r, k)
29     print(f"    {k:>4}    {m:>12.8f}    {acf_sm[k]:>12.8f}
   {'OK' if abs(m-acf_sm[k])<1e-4 else 'DIFF':>6}")
30
31 #          PART B -- FPE Criterion [5 marks]
32
33 p_range = range(0, 7); fpe_vals = []; aic_vals = []
34 for p in p_range:
35     if p == 0:
36         sigma2 = np.var(r, ddof=0)
37         fpe     = sigma2 * (n+1)/(n-1)
38         aic     = n * np.log(sigma2) + 2
39     else:
40         fit     = AutoReg(r, lags=p).fit()
41         sigma2  = fit.sigma2
42         fpe     = sigma2 * (n+p+1)/(n-p-1)
43         aic     = fit.aic

```

```

42     fpe_vals.append(fpe); aic_vals.append(aic)
43
44 fpe_p = list(p_range)[np.argmin(fpe_vals)]
45 aic_p = list(p_range)[np.argmin(aic_vals)]
46 print(f"\nPART B | FPE optimal p={fpe_p} | AIC optimal
    p={aic_p}")
47 print(f"    {'p':>3}    {'FPE':>14}    {'AIC':>10}")
48 for p, fpe, aic in zip(p_range, fpe_vals, aic_vals):
49     star = " <- FPE*" if p==fpe_p else (" " + (" <- AIC*"
        if p==aic_p else ""))
50     print(f"    {p:>3}    {fpe:>14.10f}    {aic:>10.2f}{star}")
51
52 fig, ax = plt.subplots(figsize=(10, 4))
53 ax.plot(list(p_range), fpe_vals, "bo-", lw=2, label="FPE")
54 ax.axvline(fpe_p, color="blue", linestyle="--")
55 ax2 = ax.twinx(); ax2.plot(list(p_range), aic_vals, "rs-",
    lw=2, label="AIC")
56 ax.set_xlabel("AR order p");
    ax.set_ylabel("FPE",color="blue")
57 ax2.set_ylabel("AIC",color="red"); ax.set_title("PART B --
    FPE and AIC vs AR Order: BTC")
58 ax.legend(loc="upper left"); ax2.legend(loc="upper right")
59 plt.tight_layout(); plt.show()
60
61 #          PART C -- AR Forecast [5 marks]
62
63 p_opt = fpe_p if fpe_p > 0 else 1
64 fit    = ARIMA(returns, order=(p_opt, 0, 0)).fit()
65
66 fig, ax = plt.subplots(figsize=(12, 4))
67 plot_acf(fit.resid.dropna(), lags=20, alpha=0.05, ax=ax,
    zero=False)
68 ax.set_title(f"PART C -- Residual ACF: AR({p_opt}) on BTC
    Log Returns")
69 plt.tight_layout(); plt.show()
70
71 fc = fit.forecast(steps=10)
72 print(f"\nPART C -- AR({p_opt}) 10-Day Return Forecast:")
73 for i, val in enumerate(fc, 1):
74     print(f"    Day {i:>2}: {val:+.8f}")
75 print(f"\n    Mean of 10-day forecast: {fc.mean():+.8f}
    (converges to ~0)")
76
77 # Forecast flattens near zero because:
78 # AR(p) with |phi| < 1 (stationarity): h-step forecast =
    phi^h x last_obs
79 # As h -> 10, phi^h -> 0 for any |phi| < 1
80 # Forecast converges to the unconditional mean
    (approximately 0 for returns)

```

```
79 # This is statistically correct: ARIMA cannot reliably
    predict direction beyond 3-5 days
```

M3.040 Exam Style — Integrated Module 3 Capstone **EXAM**

Marks: 25 | Time: 55 minutes | Dataset: All three (BTC, MRF, Gold)

Part A (5 marks): For all three assets (monthly), fit the best Holt-Winters variant using AIC. Print a unified AIC comparison table across all three assets and all variants.

Part B (5 marks): Generate 12-month forecasts from best models for each asset. Plot all three forecasts on a 3-panel figure. Annotate final forecast month value.

Part C (5 marks): Compute 12-month test RMSE and MAPE for all three assets. Present a final model performance table. Crown the “most predictable” asset by MAPE.

Part D (5 marks): For the most predictable asset only, implement 1-step-ahead walk-forward (min_train=24 months). Plot walk-forward accuracy and compute the proportion of steps where the model beats the naïve benchmark.

Part E (5 marks): Write a 15-line executive summary comment block covering: data used, model selection, accuracy findings, key limitations, and investment implications.

Solution

① Complete Solution

```
1 !pip install yfinance statsmodels arch -q
2 import yfinance as yf, numpy as np, pandas as pd
3 import matplotlib.pyplot as plt, warnings
4 warnings.filterwarnings('ignore')
5 from statsmodels.tsa.holtwinters import
    ExponentialSmoothing
6
7 # Data
8 tickers = {"Bitcoin": "BTC-USD", "MRF Tyres": "MRF.NS",
9            "Gold": "GC=F"}
10 data_m = {}
11 for name, ticker in tickers.items():
12     raw = yf.download(ticker, start="2018-01-01",
13                       end="2024-12-31")
14     data_m[name] =
15         raw["Close"].dropna().resample("ME").last()
16
17 train = {k: v.iloc[:-12] for k,v in data_m.items()}
18 test = {k: v.iloc[-12:] for k,v in data_m.items()}
19
20 # PART A -- Best HW Variant [5 marks]
```

```

18 variants = [
19     ("HW (trend only)",
20      {"trend":"add","seasonal":None}),
21     ("HW (add+add)",
22      {"trend":"add","seasonal":"add","seasonal_periods":12}),
23     ("HW (add+mul)",
24      {"trend":"add","seasonal":"mul","seasonal_periods":12}),
25 ]
26 all_results = []; best_fits = {}; best_fcs = {}
27
28 for asset, series_train in train.items():
29     best_aic, best_name, best_fit = np.inf, None, None
30     for vname, vparams in variants:
31         try:
32             if vparams["seasonal"] is None:
33                 model = ExponentialSmoothing(series_train,
34                                                trend="add")
35             else:
36                 model = ExponentialSmoothing(series_train,
37                                                **vparams)
38             fit = model.fit(optimized=True, disp=False)
39             all_results.append({"Asset":asset,"Variant":vname,"AIC":round(fit.aic,2)})
40             if fit.aic < best_aic:
41                 best_aic, best_name, best_fit = fit.aic, vname, fit
42         except Exception:
43             all_results.append({"Asset":asset,"Variant":vname,"AIC":np.inf})
44     best_fits[asset] = best_fit; best_fcs[asset] = best_fit.forecast(12)
45
46 aic_df =
47     pd.DataFrame(all_results).pivot(index="Variant",columns="Asset",values="AIC")
48 print("PART A -- AIC Comparison Table (lower=better)");
49 print("="*70)
50 print(aic_df.to_string()); print("="*70)
51
52 # PART B -- 3-Panel Forecast [5 marks]
53
54 fig, axes = plt.subplots(3, 1, figsize=(16, 14))
55 colors = ["orange","steelblue","gold"]
56 for ax, (asset, series), color in zip(axes,
57 data_m.items(), colors):
58     fc = best_fcs[asset]
59     ax.plot(series[-24:], color=color, linewidth=1.5,
60            label="Historical")
61     ax.plot(test[asset], color="black", linewidth=1.5,
62            linestyle="--", label="Actual Test")
63     ax.plot(fc, color="red", linewidth=2.5,
64            label="12-Month Forecast")

```

```

53     ax.annotate(f"${fc.iloc[-1]:.0f}", xy=(fc.index[-1],
54         fc.iloc[-1]),
55         xytext=(10,0), textcoords="offset points",
56         fontsize=9, color="red")
57     ax.axvline(train[asset].index[-1], color="gray",
58         linestyle=":", linewidth=1.5)
59     ax.set_title(f"{asset} -- Best HW 12-Month Forecast");
60     ax.legend(fontsize=9)
61 plt.tight_layout(); plt.show()
62
63 #          PART C -- Performance Table [5 marks]
64
65 perf_rows = []
66 for asset in tickers.keys():
67     fc_v = best_fcs[asset].values; act_v =
68         test[asset].values
69     rmse = np.sqrt(np.mean((act_v - fc_v)**2))
70     mape = np.mean(np.abs((act_v - fc_v)/act_v)) * 100
71     perf_rows.append({"Asset":asset, "RMSE":round(rmse,2), "MAPE
72         (%)":round(mape,2)})
73
74 perf_df = pd.DataFrame(perf_rows).sort_values("MAPE (%)")
75 print("\nPART C -- 12-Month Test Performance");
76 print("="*45)
77 print(perf_df.to_string(index=False)); print("="*45)
78 most_pred = perf_df.iloc[0]["Asset"]
79 print(f"\nMost predictable by MAPE: {most_pred}")
80
81 #          PART D -- Walk-Forward for Most Predictable [5
82     marks]
83 wf_data = data_m[most_pred].values
84 wf_fc_l = []; wf_act_l = []
85 for t in range(24, len(wf_data)-1):
86     tr = wf_data[0:t]
87     try:
88         if t >= 2*12:
89             model = ExponentialSmoothing(tr, trend="add",
90                 seasonal="add",
91                                     seasonal_periods=12)
92         else:
93             model = ExponentialSmoothing(tr, trend="add")
94             fit = model.fit(optimized=True, disp=False)
95             wf_fc_l.append(fit.forecast(1)[0]);
96             wf_act_l.append(wf_data[t])
97     except Exception: continue
98
99 wf_fc = np.array(wf_fc_l); wf_act = np.array(wf_act_l)
100 wf_naive = np.concatenate([[wf_data[23]],
101     wf_act[:-1]])[:len(wf_fc)]

```

```

90 beats = np.mean(np.abs(wf_act - wf_fc) < np.abs(wf_act -
    wf_naive))
91
92 fig, axes = plt.subplots(2,1, figsize=(14,8))
93 axes[0].plot(wf_act, color="steelblue", lw=1,
    label="Actual")
94 axes[0].plot(wf_fc, color="red", lw=1.2, label="HW
    Forecast")
95 axes[0].set_title(f"PART D -- Walk-Forward: {most_pred}");
    axes[0].legend()
96 axes[1].bar(range(len(wf_fc)),
    np.abs(wf_naive-wf_act) - np.abs(wf_fc-wf_act),
97 color=["green" if x>0 else "red"
98 for x in
99         (np.abs(wf_naive-wf_act)-np.abs(wf_fc-wf_act))],
    alpha=0.6)
100 axes[1].axhline(0, color="black")
101 axes[1].set_title("HW advantage over Naive each step
    (green=HW wins)")
102 plt.tight_layout(); plt.show()
103 print(f"\nPART D -- Walk-Forward: {most_pred}")
104 print(f"Steps={len(wf_fc)} | Beats naive: {beats:.1%} of
    steps")
105
106 # PART E -- Executive Summary [5 marks]
107 gold_mape = perf_df[perf_df["Asset"]=="Gold"]["MAPE
    (%)"].values[0]
108 btc_mape = perf_df[perf_df["Asset"]=="Bitcoin"]["MAPE
    (%)"].values[0]
109 mrf_mape = perf_df[perf_df["Asset"]=="MRF Tyres"]["MAPE
    (%)"].values[0]
110
111 print(f"""
112 #
113 # EXECUTIVE SUMMARY -- FORECAST MODEL EVALUATION REPORT
114 # Time Series Analytics T6725 | Module 3 Capstone
115 #
116 # SCOPE: HW exponential smoothing for 12-month price
    forecasting
117 # of Bitcoin (BTC-USD), MRF Tyres (MRF.NS), and Gold
    Futures
118 # (GC=F) using monthly price data from January 2018 to
    December 2024.
119 #
120 # MODEL SELECTION: AIC-based selection across three HW
    variants

```

```

121 # (trend-only, add seasonal, mul seasonal). All models
    use
122 # additive trend with optimised smoothing parameters
    (alpha, beta, gamma).
123 #
124 # ACCURACY (12-month hold-out test):
125 # Gold : MAPE={gold_mape:.2f}% -- {'MOST PREDICTABLE'
    if most_pred=='Gold' else 'second'}; annual demand
    cycles well-captured.
126 # MRF : MAPE={mrf_mape:.2f}% -- seasonal tyre demand
    visible; moderate accuracy.
127 # Bitcoin: MAPE={btc_mape:.2f}% -- high volatility
    limits accuracy.
128 #
129 # MOST PREDICTABLE: {most_pred} (lowest MAPE).
    Walk-forward confirms
130 # HW beats naive in {beats:.0%} of monthly forecast
    steps.
131 #
132 # LIMITATIONS: HW captures trend and seasonality but not:
133 # (a) volatility clustering -- GARCH model needed
    (Module 4);
134 # (b) structural breaks -- COVID-era dynamics may not
    repeat;
135 # (c) regime changes -- rolling window retraining
    recommended.
136 #
137 # INVESTMENT IMPLICATIONS: Gold forecast shows continued
    upward trend
138 # with seasonal buying opportunities. BTC forecast's
    wide confidence
139 # intervals suggest position sizing based on GARCH
    volatility output
140 # rather than point forecasts alone.
141 #
142 """

```

④ Real World Link

This capstone mirrors the analytical workflow of a multi-asset quantitative research team producing a quarterly forecast outlook. Part E's executive summary is the deliverable that reaches portfolio managers — it synthesises 50 lines of statistical analysis into 15 lines of actionable intelligence. The ability to write this summary clearly and concisely, with specific numbers and honest caveats, is the defining skill of a senior data scientist in finance.

Module 3 Complete

40 Problems — 10 Easy — 12 Medium — 12 Hard — 6 Exam

Range	Difficulty	Types
M3.001–M3.010	EASY	FIB, MCQ, PTO, FTE, SA
M3.011–M3.022	MEDIUM	FIB, FTE, GOR, MTF, MCQ, WFS
M3.023–M3.034	HARD	FIB, FTE, MCQ, WFS, PTO, SA
M3.035–M3.040	EXAM	Multi-part
Total	40 Problems	All 8 types

Problems	Type
M3.001, M3.002, M3.009, M3.011, M3.017, M3.021, M3.023, M3.028	Fill in the Blank
M3.006, M3.012, M3.018, M3.022, M3.024	Find the Error
M3.013	Get the Order Right
M3.014	Match the Following
M3.003, M3.004, M3.015, M3.019, M3.025, M3.030, M3.032, M3.033	MCQ / Predict Output
M3.005, M3.008, M3.027	Predict the Output
M3.016, M3.020, M3.026, M3.031	Write from Scratch
M3.007, M3.010, M3.029, M3.034	Short Answer
M3.035–M3.040	Exam Style

Ready for **Module 4** — Multivariate & Volatility: VAR, ARCH, GARCH, Conditional Heteroscedasticity